i.MX 93 BSP Manual PD24.1.1

PHYTEC Messtechnik GmbH

2024 年 10 月 23 日

Contents

1 支持的硬件 **3**
1.1 phyBOARD-Segin i.MX 93 器件 . . . 3
1.2 phyBOARD-Nash i.MX 93 器件 . . . 3

2 开始使用 **7**
2.1 下载镜像 . . . 7
2.2 将镜像写入 SD 卡 . . . 8
2.3 首次启动 . . . 10

3 构建 BSP（板级支持包） **11**
3.1 基本设置 . . . 11
3.2 下载 BSP . . . 11

4 安装操作系统 **15**
4.1 启动模式开关 (S3) . . . 15
4.2 烧写 eMMC . . . 17
4.3 RAUC . . . 20

5 开发 **21**
5.1 主机网络准备 . . . 21
5.2 从网络启动内核 . . . 23
5.3 使用 UUU 工具 . . . 24
5.4 Standalone Build preparation . . . 26
5.5 U-Boot standalone build . . . 27
5.6 Kernel standalone build . . . 29
5.7 格式化 SD 卡 . . . 31

6 设备树 (DT) **39**
6.1 介绍 . . . 39
6.2 PHYTEC i.MX 93 BSP 设备树概念 . . . 39

7 访问外设 **43**
7.1 i.MX 93 引脚复用 . . . 43
7.2 网络 . . . 44
7.3 SD/MMC 卡 . . . 46
7.4 eMMC 设备 . . . 46
7.5 GPIOs . . . 54

7.6 ADC . . . 55
7.7 LED 灯 . . . 56
7.8 I²C 总线 . . . 56
7.9 EEPROM . . . 57
7.10 RTC . . . 58
7.11 USB 主控制器 . . . 60
7.12 RS232/RS485 . . . 60
7.13 CAN FD . . . 61
7.14 phyBOARD-Segin 上的音频 . . . 62
7.15 phyBOARD-Nash 上的音频 . . . 64
7.16 视频 . . . 66
7.17 显示 . . . 66
7.18 电源管理 . . . 68
7.19 热管理 . . . 69
7.20 看门狗 . . . 70
7.21 bbnsm 电源键 . . . 71
7.22 PXP . . . 71
7.23 片上一次性可编程控制器 (OCOTP_CTRL) - eFuse . . . 71
7.24 TPM . . . 73

8 i.MX 93 M33 Core **75**
8.1 运行 M33 Core 示例 . . . 75

i.MX 93 BSP 手册	
文档标题	i.MX 93 BSP 手册
文档类型	BSP 手册
Yocto 手册	Mickledore
发布日期	2024/01/31
是... 的分支	i.MX 93 BSP 手册

下表显示了与本手册兼容的 BSP。

兼容的 BSPs	BSP 发布类型	BSP 发布日期	BSP 状态
BSP-Yocto-NXP-i.MX93-PD24.1.1	小更新	2024/05/07	已发布

本 BSP 手册指导您完成板级支持包（BSP）的安装和创建步骤，并描述如何使用 **phyBOARD-Segin i.MX 93 and phyBOARD-Nash i.MX 93 Kit** 的接口。此外，本文件还描述了如何从源代码创建 BSP 镜像。这对于需要更改默认镜像并需要以简单和可重复的方式实现这些更改的用户非常有用。此外，本手册的某些部分需要在个人计算机（主机）上执行命令。所有这些命令都假定在 Linux 操作系统上执行。

备注

本文件包含代码示例，描述如何通过串行终端与板子进行通信。代码示例的行以“host:~$”、“target:~$”或“u-boot=>”开头。这些关键字描述了命令执行的位置。只有在这些关键字之后，才需要复制实际的命令。

PHYTEC 提供各种硬件和软件文档。这包括以下内容：

- **QS Guide**：关于如何设置和启动 phyCORE 开发板的简要指南，以及构建 BSP、设备树和访问外设的简要信息。
- **Hardware Manual**：对模块系统及其配套载板的详细描述。
- **Yocto Guide**：这是一个针对 phyCORE 使用的 Yocto 版本的全面指南。该指南包含了 Yocto 的概述；介绍、安装和定制 PHYTEC BSP；如何使用 Poky 和 Bitbake 等程序；以及更多内容。
- **BSP Manual**：这是针对 phyCORE 的 BSP 版本的手册。您可以在这里找到有关如何构建 BSP、启动、更新软件、设备树和访问外设等信息。
- **Development Environment Guide**：本指南介绍了如何使用 PHYTEC 开发和准备的虚拟机（VM）主机来运行各种开发环境。指南中包含了针对 Eclipse 和 Qt Creator 的详细逐步说明，这些工具都包含在虚拟机中。此外，还有关于如何在 phyCORE 产品上运行这些程序的 Demo 项目的说明。指南还包括如何自行构建 Linux 主机 PC 的信息。
- **Pin Muxing Table**：phyCORE SOMs 附带一个引脚表（Excel 格式）。该表将显示从处理器到底板的默认信号路径。默认的设备树复用选项也将包含在内。这为开发人员提供了所有所需的信息，方便他们在开发底板或将 PHYTEC phyCORE SOM 适配到应用时进行设计。

除了这些标准手册和指南，PHYTEC 还将提供产品变更通知、应用笔记和技术笔记。这些文档将根据具体情况发布。大部分文档可以在我们产品的 https://www.phytec.de/produkte/system-on-modules/phycore-imx-91-93/#downloads 中找到。

CHAPTER 1

支持的硬件

在我们的网页上，您可以查看所有 Supported Machines 及此版本的订货号：BSP-Yocto-NXP-i.MX93-PD24.1.1，请参见 下载。

如果您在 “Supported Machines” 部分选择了特定的 **Machine Name**，您可以查看该机器下可用的 **Article Number* * 以及硬件信息的简短描述。如果您只有硬件的 **Article Number** ，可以将 **Machine Name** 下拉菜单留空，仅选择您的 **Article Number** 。现在，它应该会显示您特定硬件所需的 **Machine Name** 。

小技巧

本 BSP 手册中的终端示例仅针对 phyBOARD-Segin i.MX 93。类似的命令也可以在 phyBOARD-Nash i.MX 93 上执行。

1.1 phyBOARD-Segin i.MX 93 器件

1.2 phyBOARD-Nash i.MX 93 器件

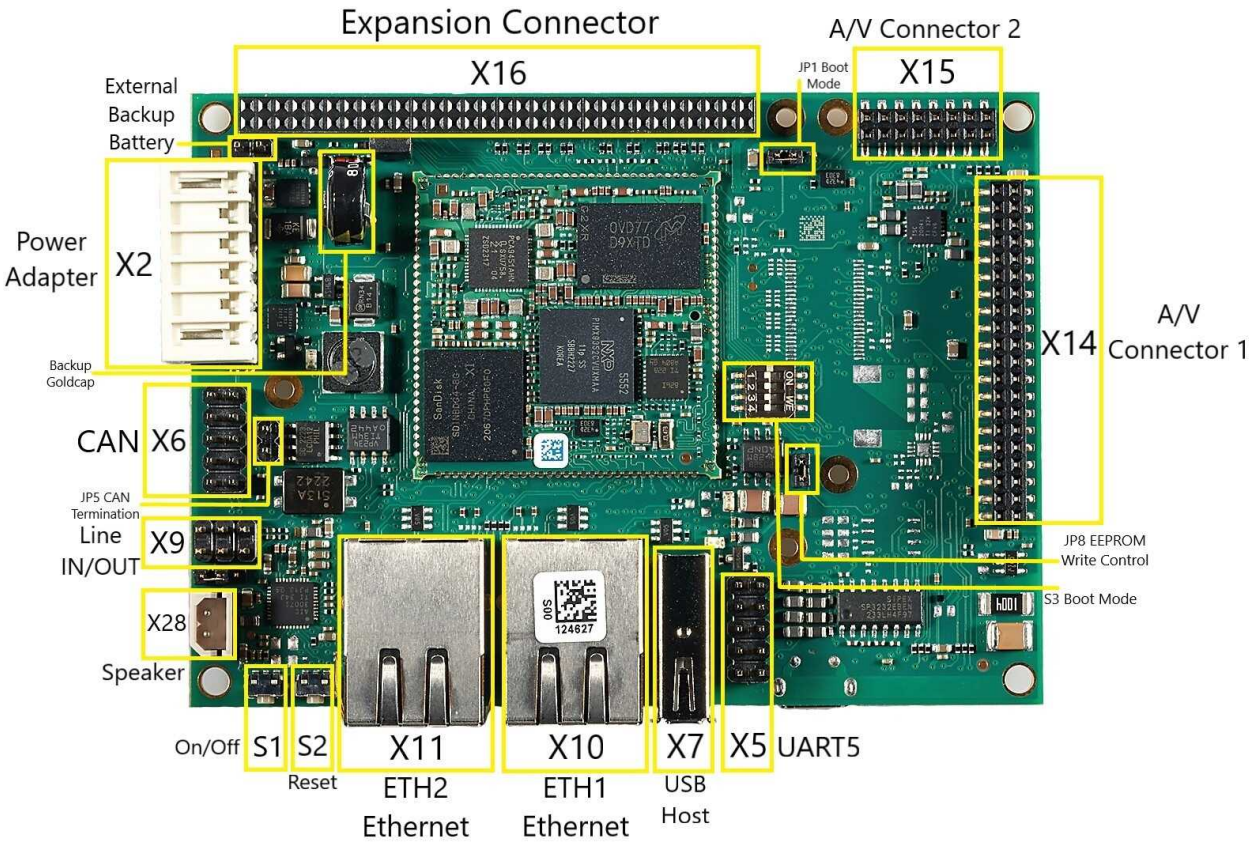


图 1: **phyBOARD-Segin i.MX 93 器件（顶部）**

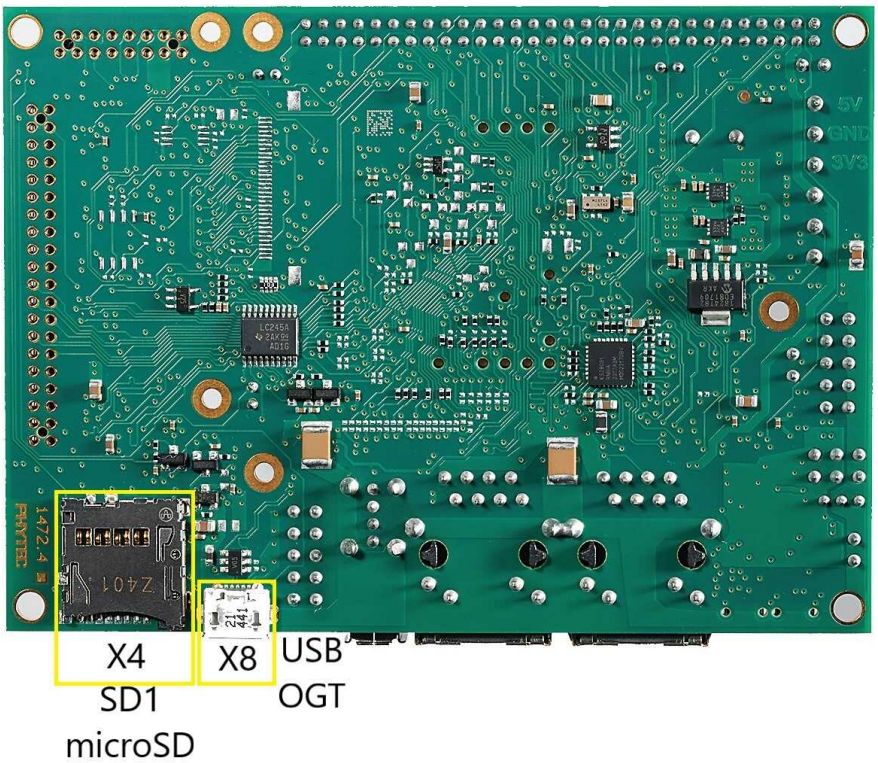


图 2: **phyBOARD-Segin i.MX 93 器件（底部）**

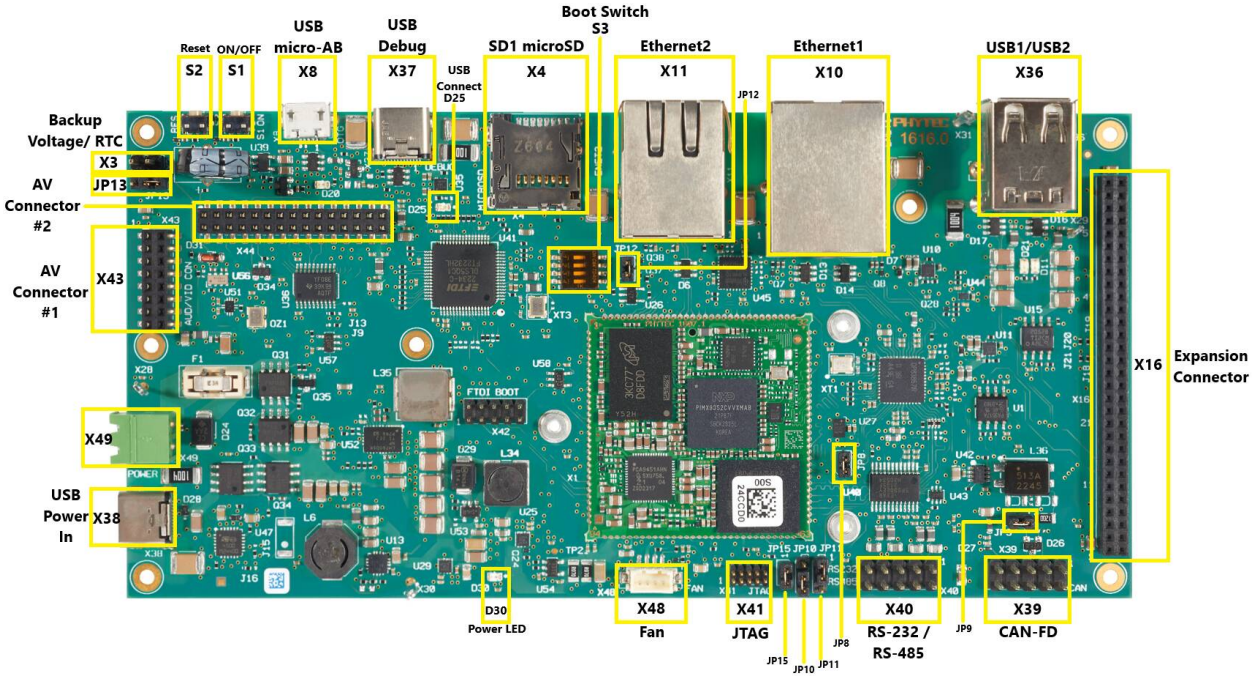


图 3: **phyBOARD-Nash i.MX 93 器件（顶部）**

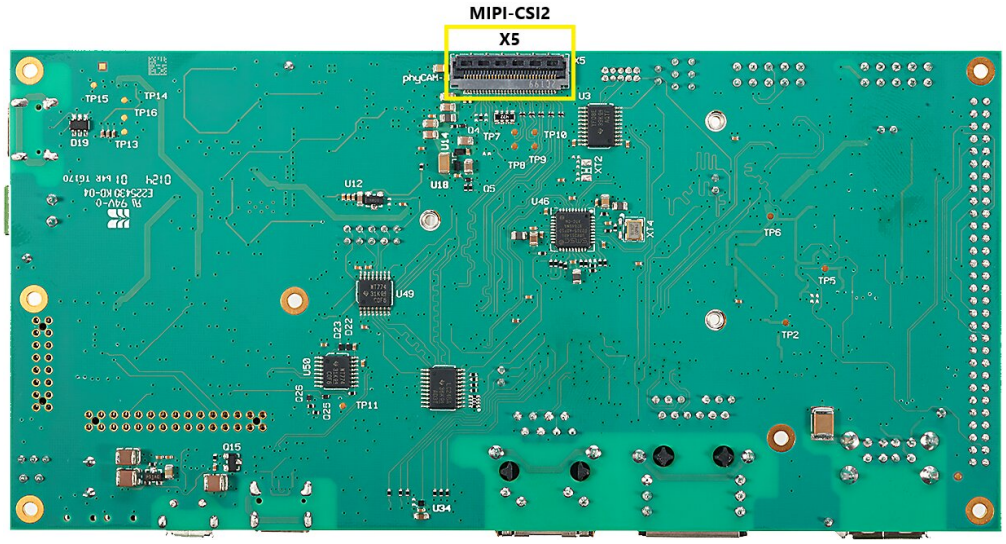


图 4: **phyBOARD-Nash i.MX 93 器件（底部）**

CHAPTER 2

开始使用

该 **phyBOARD-Segin i.MX 93 and phyBOARD-Nash i.MX 93 Kit** 随附预先刷写的 SD 卡。它包含 phytec-qt6demo-image ，可以直接用作启动源。默认情况下，eMMC 仅编程了 U-Boot。您可以从 PHYTEC 下载服务器 获取所有源代码。本章将解释如何将 BSP 镜像刷写到 SD 卡以及如何启动开发板。

有几种方法可以将镜像写入 SD 卡或甚至 eMMC。最显著的是使用 Linux 命令行工具 `dd` 进行简单的顺序写入。另一种方法是使用 PHYTEC 的系统初始化程序 partup ，这使得格式化更复杂的系统变得特别简单。您可以从其发布页面获取 partup 的预构建 Linux 二进制文件 。还请阅读 partup 的自述文件 以获取安装说明。

2.1 下载镜像

该镜像包含所有必要的文件，并确保分区和任何原始数据被正确写入。可以从 PHYTEC 下载服务器 下载 partup 包和可以使用 `dd` 刷写的 WIC 镜像。

从下载服务器获取 partup 包或 WIC 镜像：

```
host:~$ wget https://download.phytec.de/Software/Linux/BSP-Yocto-i.MX93/BSP-Yocto-NXP-i.MX93-PD24.1.1/
↪images/ampliphy-vendor-wayland/phyboard-segin-imx93-2/phytec-qt6demo-image-phyboard-segin-imx93-2.partup
host:~$ wget https://download.phytec.de/Software/Linux/BSP-Yocto-i.MX93/BSP-Yocto-NXP-i.MX93-PD24.1.1/
↪images/ampliphy-vendor-wayland/phyboard-segin-imx93-2/phytec-qt6demo-image-phyboard-segin-imx93-2.wic.xz
```

备注

对于 eMMC，更复杂的分区方案或甚至只是大型镜像，我们建议使用 partup 软件包，因为它在写入速度上比 `dd` 更快，并且允许对闪存设备进行更灵活的配置。

2.2 将镜像写入 SD 卡

警告

要创建可启动的 SD 卡，您必须在 Linux 主机 PC 上拥有 root 权限。在指定目标设备时请务必小心！所选设备上的所有文件将立即被删除，而不会有任何进一步的确认！

选择错误的设备可能会导致 **数据丢失**，例如，可能会擦除您当前在主机 PC 上运行的系统！

2.2.1 寻找正确的设备

要创建可启动的 SD 卡，您首先必须找到 SD 卡及其可能的分区的正确设备名称。如果 SD 卡的任何分区已挂载，请在开始将镜像复制到 SD 卡之前卸载这些分区。

1. 为了获取正确的设备名称，请移除您的 SD 卡并执行：

```
host:~$ lsblk
```

2. 现在插入你的 SD 卡，然后再次执行命令：

```
host:~$ lsblk
```

3. 比较两个输出，以查找第二个输出中列出的新设备名称。这些是 SD 卡的设备名称（如果 SD 卡已格式化，则包括设备和分区）。
4. 为了验证找到的设备名称，请执行命令 `sudo dmesg`。在其输出的最后几行中，您应该也能找到设备名称，例如 `/dev/sde` 或 `/dev/mmcblk0` （具体取决于您的系统）。

或者，您可以使用您选择的图形程序，例如 GNOME Disks 或 KDE Partition Manager 来找到正确的设备。

现在您已经获得了正确的设备名称，例如 `/dev/sde`，您可以查看必须在格式化 SD 卡时卸载的分区。在这种情况下，您还会在输出中找到带有附加数字的设备名称（例如 `/dev/sde1`）。这些代表分区。一些 Linux 发行版在设备插入时会自动挂载分区。然而，在写入之前，必须卸载这些分区，以避免数据损坏。

卸载所有这些分区，例如：

```
host:~$ sudo umount /dev/sde1
host:~$ sudo umount /dev/sde2
```

现在，SD 卡已经准备好可以使用 `partup` 、`dd` 或 `bmap-tools` 来写入镜像。

2.2.2 使用 bmap-tools

准备 SD 卡的一种方法是使用 bmap-tools 。Yocto 会自动为 WIC 镜像创建一个块映射文件（`<IMAGENAME>-<MACHINE>.wic.bmap` ），该文件描述了镜像内容并包含数据完整性的校验和。*bmaptool* 已被多种 Linux 发行版打包。对于基于 Debian 的系统，可以通过以下命令安装：

```
host:~$ sudo apt install bmap-tools
```

通过以下命令将 WIC 镜像烧录到 SD 卡：

```
host:~$ bmaptool copy phytec-qt6demo-image-phyboard-segin-imx93-2?(.rootfs).wic?(.xz) /dev/<your_device>
```

将 <your_device> 替换为您之前找到的实际 SD 卡设备名称，并确保将文件 `<IMAGENAME>-<MACHINE>.wic.bmap` 与常规 WIC 镜像文件放在一起，以便 bmaptool 知道哪些块需要写入，哪些块需要跳过。

警告

bmaptool 仅覆盖 SD 卡上镜像数据所在的区域。这意味着在写入镜像后，之前写入的 U-Boot 环境可能仍然可用。

2.2.3 使用 partup

使用 partup 写入 SD 卡只需一个命令：

```
host:~$ sudo partup install phytec-qt6demo-image-phyboard-segin-imx93-2?(.rootfs).partup /dev/<your_device>
```

确保将 <your_device> 替换为您之前找到的实际设备名称。

关于 partup 的进一步使用说明，请参阅其 官方文档网站 。

警告

使用 resize2fs 版本 1.46.6 及更早版本的主机系统（例如 Ubuntu 22.04）无法将使用 Yocto Mickledore 或更新版本创建的 partup 软件包写入 SD 卡。这是由于 resize2fs 中的一个新默认选项导致的不兼容性。有关详细信息，请参阅 发行说明 。

备注

partup 的优点在于可以清除 MMC 用户区中的特定原始区域，这在我们提供的 partup 软件包中用于擦除任何现有的 U-Boot 环境。这是一个已知的问题，*bmaptool* 并不能解决，如前一章所提到的。

partup 相较于其他刷写工具的另一个主要优势是，它允许配置特定于 MMC 的部分，比如在写入时可以直接写入 eMMC 引导分区，而无需调用多个其他命令。

2.2.4 使用 dd

在卸载所有 SD 卡的分区后，您可以创建可启动的 SD 卡。

一些 PHYTEC BSP 生成未压缩的镜像（文件名扩展名为 *.wic），而另一些则生成压缩的镜像（文件名扩展名为 *.wic.xz）。

要写入未压缩的镜像（*.wic），请使用以下命令：

```
host:~$ sudo dd if=phytec-qt6demo-image-phyboard-segin-imx93-2?(.rootfs).wic of=/dev/<your_device> bs=1M␣
↪conv=fsync status=progress
```

或者要写入压缩后的镜像（*.wic.xz），请使用以下命令：

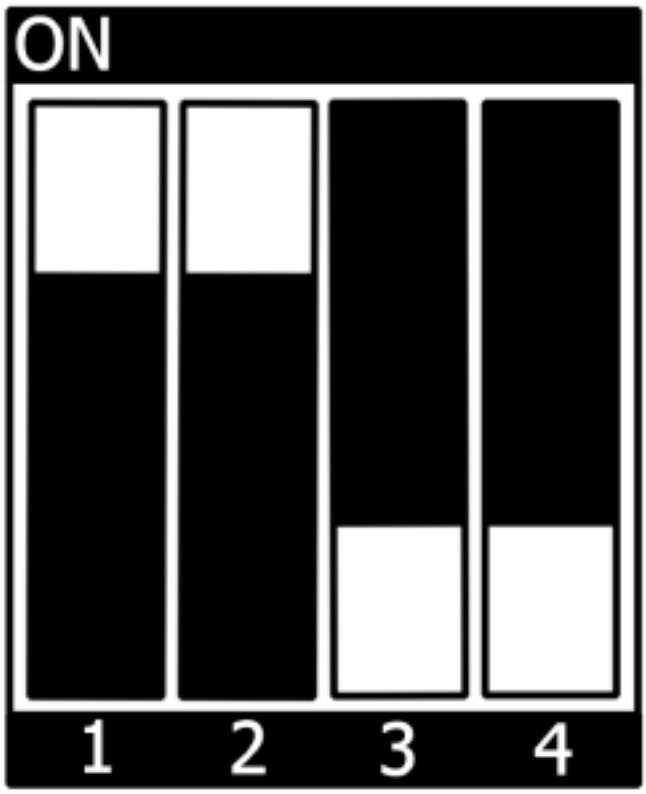
```
host:~$ xzcat phytec-qt6demo-image-phyboard-segin-imx93-2?(.rootfs).wic.xz | sudo dd of=/dev/<your_device>␣
↪bs=1M conv=fsync status=progress
```

再次确保将 <your_device> 替换为之前找到的实际设备名称。

参数 `conv=fsync` 强制在 `dd` 返回之前对设备进行同步操作。这确保所有数据块都已写入 SD 卡，而没有任何数据留在内存中。参数 `status=progress` 将打印出有关已复制和仍需复制的数据量的信息，直到操作完成。

2.3 首次启动

- 要从 SD 卡启动，*bootmode switch (S3)* 需要设置为以下位置：



- 插入 SD 卡
- 将目标设备的调试终端与您的主机连接。使用 UART1 console on PEB-EVAL-01 for **phyBOARD-Segin** and X-37 USB-C debug for **phyBOARD-Nash**。
- 给开发板通电
- 以 115200 波特率和 8N1 格式打开串口/USB 端口（您应该在终端上看到 u-boot/linux 启动信息）

CHAPTER 3

构建 BSP（板级支持包）

本节将指导您通过使用 Yocto 和 phyLinux 脚本进行 i.MX 93 BSP 的一般构建过程。如需有关我们的 meta-layer 或 Yocto 的一般信息，请访问：Yocto Reference Manual (mickledore) 。

3.1 基本设置

如果您从未在您的计算机上使用 Yocto 创建过 Phytec BSP，您应该仔细查看 Yocto Reference Manual (mickledore) 中的 BSP Workspace 安装章节。

3.2 下载 BSP

获取 BSP 源代码有两种方式。您可以从我们的下载页面下载完整的 BSP 源代码：BSP-Yocto-i.MX93 ；或者您可以使用 Yocto 自行获取并构建。这在您想要进行自定义时特别有用。

phyLinux 脚本是一个用于 PHYTEC Yocto BSP 的基本管理工具，使用 Python 编写。它主要是一个辅助工具，用于初始化 BSP。

- 创建一个新的项目文件夹，获取 phyLinux，并使脚本可执行：

```
host:~$ mkdir ~/yocto
host:~$ cd yocto/
host:~/yocto$ wget https://download.phytec.de/Software/Linux/Yocto/Tools/phyLinux
host:~/yocto$ chmod +x phyLinux
```

> **警告**
>
> 一个干净的文件夹很重要，因为 phyLinux 会清理其工作目录。从一个不为空的目录调用 phyLinux 将会导致警告。

- 运行 phyLinux：

```
host:~/yocto$ ./phyLinux init
```

备注

在首次初始化时，phyLinux 脚本会要求您在 /usr/local/bin 目录中安装 Repo 工具。

- 在执行初始化命令时，您需要选择您的处理器平台（SoC）、PHYTEC 的 BSP 版本号以及您正在使用的硬件。

备注

如果您无法根据菜单中提供的信息识别您的开发板，请查看产品的发票。并查看 our BSP 。

- 也可以通过命令行参数直接传递这些信息：

```
host:~/yocto$ DISTRO=ampliphy-vendor-wayland MACHINE=phyboard-segin-imx93-2 ./phyLinux init -p imx93 -
↪r BSP-Yocto-NXP-i.MX93-PD24.1.1
```

在执行 init 命令后，phyLinux 将打印一些重要的说明。例如，它将打印您的 git 身份、选择的 SOC 和 BSP 版本，以及构建过程下一步的信息。

3.2.1 开始构建过程

- 设置 Shell 环境变量：

```
host:~/yocto$ source sources/poky/oe-init-build-env
```

备注

在每次打开新的用于编译的终端时，都需要执行。

- 当前的工作目录应该更改为 build/。
- 打开主配置文件，接受 GPU 和 VPU 二进制许可证协议。通过取消注释相应的行来完成此操作，如下所示。

```
host:~/yocto/build$ vim conf/local.conf
# Uncomment to accept NXP EULA
# EULA can be found under ../sources/meta-freescale/EULA
ACCEPT_FSL_EULA = "1"
```

- 构建您的镜像：

```
host:~/yocto/build$ bitbake phytec-qt6demo-image
```

备注

对于第一次构建，我们建议从我们的较小的非图形化镜像 phytec-headless-image 开始，以查看一切是否正常工作。

```
host:~/yocto/build$ bitbake phytec-headless-image
```

第一次编译过程在现代的 Intel Core i7 处理器上大约需要 40 分钟。所有后续的构建将使用缓存，应该大约需要 3 分钟。

3.2.2 BSP 镜像

All images generated by Bitbake are deployed to `~/yocto/build/deploy*/images/<machine>`. The following list shows for example all files generated for the phyboard-segin-imx93-2 machine:

- **u-boot.bin**: 编译后的 U-boot bootloader 二进制文件。不是最终的 bootloader 镜像！
- **oftree**: 默认内核设备树
- **u-boot-spl.bin**: 二级程序加载器 (SPL)
- **bl31-imx93.bin**: ARM 可信固件二进制文件
- **lpddr4_dmem_1d_v202201.bin, lpddr4_dmem_2d_v202201.bin, lpddr4_imem_1d_v202201.bin, lpddr4_imem_2d_v202201.bin**: DDR PHY 固件镜像
- **imx-boot**：由 imx-mkimage 构建的 bootloader，包括 SPL、U-Boot、ARM 可信固件和 DDR 固件。这是最终的可引导 bootloader 镜像。
- **Image**: Linux 内核镜像
- **Image.config**: 内核配置文件
- **imx93-phyboard-*.dtb**: 内核设备树文件
- **imx93-phy*.dtbo**: 内核设备树 overlay 文件
- **phytec-*.tar.gz**: bitbake-image 编译生成的 root 文件系统。
  - **phytec-qt6demo-image-phyboard-*-imx93-*.tar.gz**：在使用 bitbake-build 编译 `phytec-qt6demo-image` 时
  - **phytec-headless-image-phyboard-*-imx93-*.tar.gz**：在使用 bitbake-build 编译 `phytec-headless-image` 时
- **phytec-*.wic.xz**：bitbake-image 编译生成给的压缩可引导 SD 卡镜像，包括 bootloader、设备树二进制文件（DTB）、内核和根文件系统。
  - **phytec-qt6demo-image-phyboard-*-imx93-*.wic.xz**：在使用 bitbake-build 编译 `phytec-qt5demo-image` 时
  - **phytec-headless-image-phyboard-*-imx93-*.wic.xz**：在使用 bitbake-build 编译 `phytec-headless-image` 时
- **imx93-11x11-evk_m33_*.bin**，Cortex-M33 MCU 的 Demo 应用程序的二进制文件；可以通过 U-Boot 或 Linux 手动加载和启动

CHAPTER 4

安装操作系统

4.1 启动模式开关 (S3)

小技巧

底板版本：

- phyBOARD-Segin-i.MX 93: 1472.5
- phyBOARD-Nash-i.MX 93: 1616.0

该 |sbc| 具有一个开关，配有四个可单独切换的位，用于选择 phyCORE-i.MX 93 默认的启动源。

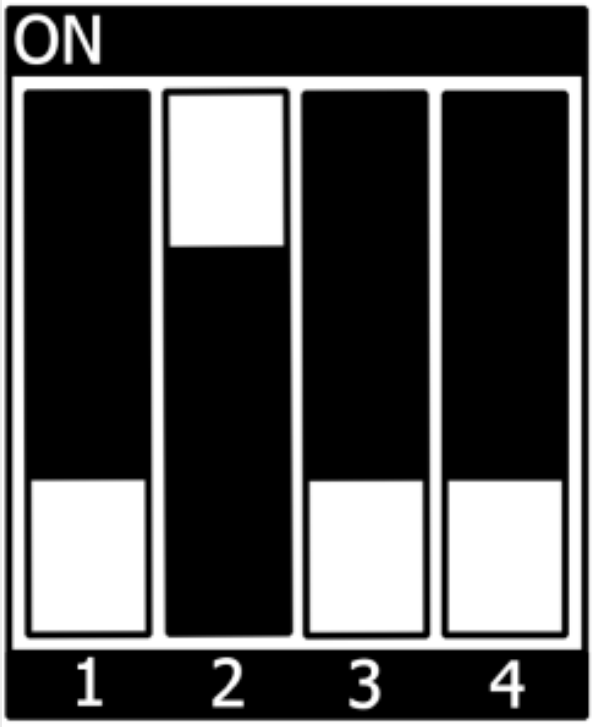


图 1: eMMC

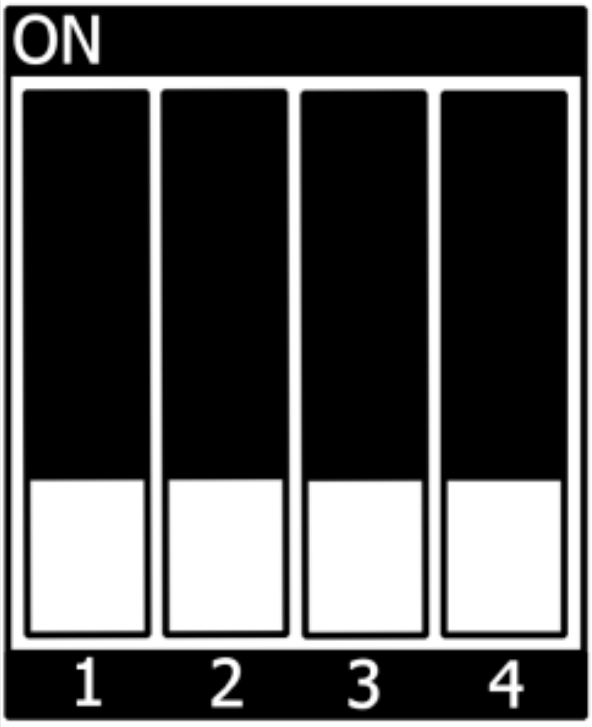


图 2: 内部 fuse

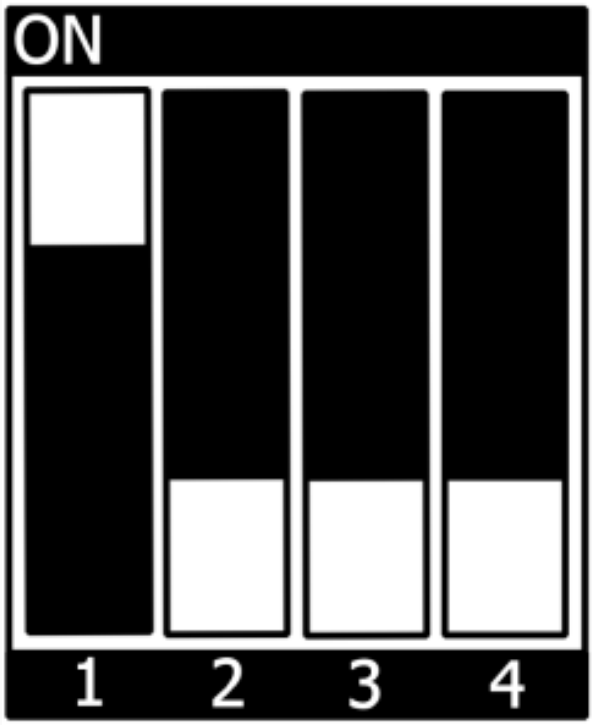


图 3: USB 串口下载

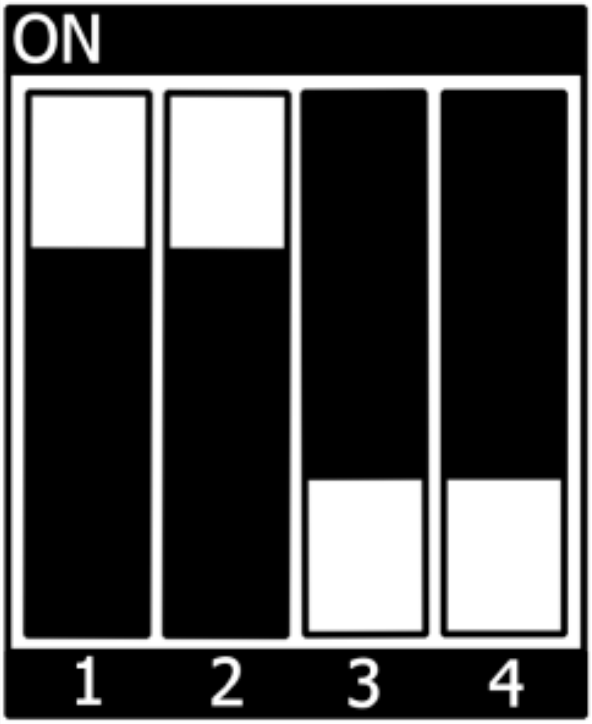


图 4: SD 卡

4.2 烧写 eMMC

要从 eMMC 启动，请确保 BSP 镜像已正确烧写到 eMMC，并且*bootmode switch (S3)* 设置为 **eMMC**。

警告

当 eMMC 和 SD 卡上烧录了相同（完全相同）的镜像时，启动分区的 UUID 也是相同的。如果在启动时连接了 SD 卡，这会导致非确定性的行为，因为 Linux 会根据 UUID 挂载启动分区。

```
target:~$ blkid
```

可以运行以检查当前设置是否受到影响。如果 mmcblk0p1 和 mmcblk1p1 具有相同的 UUID，则该设置受到影响。

4.2.1 从 SD 卡烧写 eMMC

如果没有可用的网络，您可以通过 SD 卡更新 eMMC。为此，您只需在 SD 卡上准备一个可用的镜像文件（`*.wic`）。由于镜像文件相当大，您需要扩展 SD 卡以使用其全部空间（如果之前没有扩展的话）。有关如何扩展 SD 卡，请参见 “调整 ext4 根文件系统的大小”。

或者，将一个 partup 包刷入 SD 卡，如*Getting Started* 中所述。这将确保 SD 卡的全部空间被使用。

在目标设备上通过 SD 卡在 Linux 中烧写 eMMC

您可以在 Linux 上烧写 eMMC。您只需在 SD 卡上保存一个 partup 包或 WIC 镜像即可。

- 在 SD 卡上显示您保存的 partup 包或 WIC 镜像或 WIC.XZ 镜像文件：

```
target:~$ ls
phytec-qt6demo-image-phyboard-segin-imx93-2.partup
phytec-qt6demo-image-phyboard-segin-imx93-2.wic.xz
```

- 显示可用的 MMC 设备列表：

```
target:~$ ls /dev | grep mmc
mmcblk1
mmcblk1p1
mmcblk1p2
mmcblk0
mmcblk0boot0
mmcblk0boot1
mmcblk0p1
mmcblk0p2
mmcblk0rpmb
```

- eMMC 设备的特征是它包含两个 boot 分区：(mmcblk0 **boot0**; mmcblk0 **boot1**)
- 使用 partup 将镜像写入 phyCORE-i.MX 93 的 eMMC (/dev/mmcblk0 **不带**分区)：

```
target:~$ partup install phytec-qt6demo-image-phyboard-segin-imx93-2.partup /dev/mmcblk0
```

小技巧

强烈建议使用 partup，因为它更易于使用，并且可以充分利用 eMMC 设备的容量，自动调整分区。

备注

或者，可以使用 `dd` 命令。

对于未压缩的 WIC 镜像（*.wic）：

```
target:~$ dd if=phytec-qt6demo-image-phyboard-segin-imx93-2.wic of=/dev/mmcblk0 bs=1M conv=fsync␣
↪status=progress
```

对于压缩的 WIC 镜像（*.wic.xz）：

```
target:~$ zstdcat phytec-qt6demo-image-phyboard-segin-imx93-2.wic.xz | sudo dd of=/dev/mmcblk0␣
↪bs=1M conv=fsync status=progress
```

请注意，在使用 `dd` 进行刷写时，root 分区并没有使用全部空间。

- 在完成写入后，您的板子可以从 eMMC 启动。

警告

在此之前，您需要将*bootmode switch (S3)* 配置为 eMMC。

4.2.2 从网络烧写 eMMC

i.MX 93 板具有以太网连接器，可以通过网络进行更新。确保正确设置开发主机。IP 需要设置为 192.168.3.10，子网掩码为 255.255.255.0，并且需要有一个 TFTP 服务器。从高层次来看，eMMC 设备就像 SD 卡。因此，可以直接将 **WIC 镜像**（`<name>.wic` ）从 Yocto 构建系统烧写到 eMMC。该镜像包含 bootloader、内核、设备树、设备树覆盖和根文件系统。

备注

一些 PHYTEC 的 BSP 会生成压缩的 `.wic.xz` 镜像。在这种情况下，必须先对压缩镜像进行解压缩。

```
host:~$ unxz phytec-qt6demo-image-phyboard-segin-imx93-2.wic.xz
```

通过网络在目标设备上使用 Linux 烧写 eMMC

您可以从目标设备更新 eMMC。

小技巧

需要一个有效的网络！*Setup Network Host*

在主机上获取一个未压缩的镜像，并通过网络使用 ssh 将其发送到目标设备的 eMMC，使用一行命令：

```
target:~$ ssh <USER>@192.168.3.10 "dd if=<path_to_file>/phytec-qt6demo-image-phyboard-segin-imx93-2.wic" |␣
↪dd of=/dev/mmcblk0 bs=1M conv=fsync status=progress
```

在主机上通过网络烧写 eMMC 的 Linux 方法

也可以从您的 Linux 主机将操作系统安装到 eMMC 上。和之前一样，您需要在主机上准备一个完整的镜像。

小技巧

需要一个有效的网络！*Setup Network Host*

显示主机上可用的镜像文件：

```
host:~$ ls
phytec-qt6demo-image-phyboard-segin-imx93-2.wic
```

通过网络使用 `dd` 命令结合 ssh 将镜像发送到您设备的 eMMC：

```
host:~$ dd if=phytec-qt6demo-image-phyboard-segin-imx93-2.wic bs=1M status=progress | ssh root@192.168.3.11
↪"dd of=/dev/mmcblk0 conv=fsync"
```

4.2.3 通过网络从正在运行的 U-Boot 烧写 eMMC U-Boot 镜像

从 U-Boot 单独更新 U-Boot 镜像 imx-boot 也是可能的。如果 eMMC 上的 bootloader 位于 eMMC 的 user 区域，可以使用此功能。

小技巧

需要一个有效的网络！*Setup Network Host*

通过 tftp 将镜像加载到 RAM 中，然后写入 eMMC：

```
u-boot=> tftp ${loadaddr} imx-boot
u-boot=> setexpr nblk ${filesize} / 0x200
u-boot=> mmc dev 0
u-boot=> mmc write ${loadaddr} 0x40 ${nblk}
```

提示

十六进制值表示偏移量，单位为 512 字节块的倍数。请参阅偏移表 以获取相应 SoC 的正确值。

4.2.4 从 USB 烧写 eMMC

在 Linux 中从 USB 烧写 eMMC

这些步骤将展示如何在 Linux 上使用 USB 优盘烧写 eMMC。您只需要一个保存在 USB 优盘上的完整镜像和一个可引导的 WIC 镜像（例如 phytec-qt6demo-image-phyboard-segin-imx93-2.wic）。将*bootmode switch (S3)* 设置为 SD 卡启动。

- 插入并挂载 USB 优盘：

```
[   60.458908] usb-storage 1-1.1:1.0: USB Mass Storage device detected
[   60.467286] scsi host0: usb-storage 1-1.1:1.0
[   61.504607] scsi 0:0:0:0: Direct-Access                                8.07 PQ: 0 ANSI: 2
[   61.515283] sd 0:0:0:0: [sda] 3782656 512-byte logical blocks: (1.94 GB/1.80 GiB)
```

(续下页)

(接上页)

```
[   61.523285] sd 0:0:0:0: [sda] Write Protect is off
[   61.528509] sd 0:0:0:0: [sda] No Caching mode page found
[   61.533889] sd 0:0:0:0: [sda] Assuming drive cache: write through
[   61.665969]  sda: sda1
[   61.672284] sd 0:0:0:0: [sda] Attached SCSI removable disk
target:~$ mount /dev/sda1 /mnt
```

- 现在显示您在 USB 优盘上保存的图像文件：

```
target:~$ cd /mnt
target:~$ ls
phytec-qt6demo-image-phyboard-segin-imx93-2.wic
```

- 显示可用的 MMC 设备列表：

```
target:~$ ls /dev | grep mmc
mmcblk1
mmcblk1p1
mmcblk1p2
mmcblk0
mmcblk0boot0
mmcblk0boot1
mmcblk0p1
mmcblk0p2
mmcblk0rpmb
```

- eMMC 设备的特征是它包含两个 boot 分区：(mmcblk0 **boot0**; mmcblk0 **boot1**)
- 将镜像写入 phyCORE-i.MX 93 eMMC (/dev/mmcblk0，无分区)：

```
target:~$ dd if=phytec-qt6demo-image-phyboard-segin-imx93-2.wic of=/dev/mmcblk0 bs=1M conv=fsync␣
↪status=progress
```

- 在完成写入后，您的板子可以从 eMMC 启动。

> **小技巧**
>
> 在此之前，您需要将*bootmode switch (S3)* 配置为 **eMMC**。

4.3 RAUC

RAUC 功能支持已添加到 meta-ampliphy 中。它控制着使用新固件更新设备的过程。这包括更新 Linux 内核、设备树和根文件系统。PHYTEC 编写了一份在线手册，介绍我们如何将 RAUC 集成到我们的 BSP 中：L-1006e.A5 RAUC 更新与设备管理手册。

CHAPTER 5

开发

5.1 主机网络准备

为了在 bootloader 中执行涉及网络的各种任务，需要设置一些主机服务。在开发主机上，必须安装和配置 TFTP、NFS 和 DHCP 服务器。启动以太网所需的工具如下：

```
host:~$ sudo apt install tftpd-hpa nfs-kernel-server kea
```

5.1.1 TFTP 服务器设置

- 首先，创建一个目录来存储 TFTP 文件：

  ```
  host:~$ sudo mkdir /srv/tftp
  ```

- 然后将您的 BSP 镜像文件复制到此目录，并确保其他用户对 tftp 目录中的所有文件具有读取权限，否则它们将无法从目标访问。

  ```
  host:~$ sudo chmod -R o+r /srv/tftp
  ```

- 您还需要为相应的接口配置一个静态 IP 地址。PHYTEC 评估板的默认 IP 地址是 192.168.3.11。设置主机地址为 192.168.3.10，子网掩码为 255.255.255.0 是一个不错的选择。

  ```
  host:~$ ip addr show <network-interface>
  ```

  将 <network-interface> 替换为您配置并希望连接到开发板的网络接口。您可以通过不指定网络接口来显示所有网络接口。

- 您收到的消息应包含以下内容：

  ```
  inet 192.168.3.10/24 brd 192.168.3.255
  ```

- 创建或编辑 `/etc/default/tftpd-hpa` 文件：

```
# /etc/default/tftpd-hpa

TFTP_USERNAME="tftp"
TFTP_DIRECTORY="/srv/tftp"
TFTP_ADDRESS=":69"
TFTP_OPTIONS="-s -c"
```

- 将 TFTP_DIRECTORY 设置为您的 TFTP 服务器根目录
- 将 TFTP_ADDRESS 设置为服务器正在监听的主机地址（设置为 0.0.0.0:69 以监听所有本地 IP）。
- 设置 TFTP_OPTIONS，以下命令显示可用的选项：

```
host:~$ man tftpd
```

- 重新启动服务以应用配置更改：

```
host:~$ sudo service tftpd-hpa restart
```

现在将开发板的以太网端口连接到您的主机系统。我们还需要在嵌入式板和 TFTP 服务器之间建立网络连接。服务器的 IP 地址应设置为 192.168.3.10，子网掩码为 255.255.255.0。

NFS 服务器设置

- 创建一个 NFS 目录：

```
host:~$ sudo mkdir /srv/nfs
```

- NFS 服务器并不限制于某个特定的文件系统位置，因此在大多数发行版中，我们只需修改文件 `/etc/exports` ，并将我们的根文件系统导出到嵌入式网络。在这个示例文件中，整个目录被导出，而开发主机的“实验室网络”地址为 192.168.3.10。IP 地址需要根据本地需求进行调整：

```
/srv/nfs 192.168.3.0/255.255.255.0(rw,no_root_squash,sync,no_subtree_check)
```

- 现在 NFS 服务器需要再次读取 `/etc/exportfs` 文件：

```
host:~$ sudo exportfs -ra
```

DHCP 服务器设置

- 创建或编辑 `/etc/kea/kea-dhcp4.conf` 文件；使用内部子网示例。将 <network-interface> 替换为物理网络接口的名称：

```
{
  "Dhcp4": {
    "interfaces-config": {
      "interfaces": [ "<network-interface>/192.168.3.10" ]
    },
    "lease-database": {
      "type": "memfile",
      "persist": true,
      "name": "/tmp/dhcp4.leases"
    },
    "valid-lifetime": 28800,
    "subnet4": [{
        "id": 1,
        "next-server": "192.168.3.10",
```

（续下页）

(接上页)

```
        "subnet": "192.168.3.0/24",
        "pools": [
          { "pool": "192.168.3.1 - 192.168.3.255" }
        ]
    }]
  }
}
```

警告

在创建子网时请小心，因为这可能会干扰公司网络政策。为了安全起见，请使用不同的网络，并通过 `interfaces` 配置选项指定该网络。

- 现在 DHCP 服务器需要重新读取 `/etc/kea/kea-dhcp4.conf` 文件：

```
host:~$ sudo systemctl restart kea-dhcp4-server
```

当您启动/重启主机 PC 时，如果没有按照 kea-dhcp4 配置中指定的网络接口处于活动状态，kea-dhcp4-server 将无法启动。请确保在连接接口时启动/重启 systemd 服务。

5.2 从网络启动内核

从网络启动意味着通过 TFTP 加载内核和设备树，并通过 NFS 加载根文件系统。引导加载程序本身必须已经从另一个可用的启动设备加载。

5.2.1 在主机上放置网络启动的镜像

- 将内核镜像复制到您的 tftp 目录中：

```
host:~$ cp Image /srv/tftp
```

- 将设备树复制到您的 tftp 目录：

```
host:~$ cp oftree /srv/tftp
```

- 将您想要使用的所有覆盖文件复制到您的 tftp 目录中：

```
host:~$ cp *.dtbo /srv/tftp
```

- 确保其他用户对 tftp 目录中的所有文件具有读取权限，否则它们将在目标设备上无法访问：

```
host:~$ sudo chmod -R o+r /srv/tftp
```

- 将根文件系统提取到您的 NFS 目录中：

```
host:~$ sudo tar -xvzf phytec-qt6demo-image-phyboard-segin-imx93-2.tar.gz -C /srv/nfs
```

备注

请确保使用 sudo 提权，以保留正确的所有权。

5.2.2 设置 Netboot 的 bootenv.txt 文件

在您的 tftp 目录中创建一个 bootenv.txt 文件，并将以下变量写入其中。

```
bootfile=Image
fdt_file=oftree
nfsroot=/srv/nfs
overlays=<overlayfilenames>
```

<overlayfilenames> 必须替换为您想要使用的设备树覆盖文件名。用空格分隔文件名。例如：

```
overlays=example-overlay1.dtbo example-overlay2.dtbo
```

小技巧

所有支持的设备树 overlay 在*device tree* 章节中。

5.2.3 目标上的网络设置

要自定义目标以太网配置，请按照此处的说明进行操作：*Network Environment Customization*

5.2.4 从开发板启动

将板子启动到 U-boot 提示符，并按任意键暂停。

- 要从网络启动，请调用：

```
u-boot=> run netboot
```

5.3 使用 UUU 工具

NXP 的通用更新工具（UUU-Tool）是一款在主机上执行的软件，用于通过 SDP（串行下载协议）加载并运行板上的 bootloader。有关详细信息，请访问 https://github.com/nxp-imx/mfgtools 或下载 官方 UUU 工具文档。

5.3.1 主机准备以使用 UUU 工具

- 请按照 https://github.com/nxp-imx/mfgtools#linux 上的说明进行操作。
- 如果你是从源代码构建 UUU，请将其添加到 `PATH` 中：

  这个 BASH 命令只是暂时将 UUU 添加到 `PATH` 中。要永久添加，请将此行添加到 `~/.bashrc` 中。

```
export PATH=~/mfgtools/uuu/:"$PATH"
```

- 设置 udev 规则（在 `uuu -udev` 中有详细说明）：

```
host:~$ sudo sh -c "uuu -udev >> /etc/udev/rules.d/70-uuu.rules"
host:~$ sudo udevadm control --reload
```

5.3.2 获取镜像

从我们的服务器下载 imx-boot，或者从你的 Yocto 构建目录中的 build/deploy/images/phyboard-segin-imx93-2/获取它。要将 wic 镜像烧写到 eMMC，你还需要 phytec-qt6demo-image-phyboard-segin-imx93-2.wic。

5.3.3 目标准备

将*bootmode switch (S3)* 设置为 **USB 串口下载**。同时，将 USB 端口*X8 (USB micro/OTG connector)* 连接到您的主机。

5.3.4 通过 UUU 工具启动 bootloader

执行并启动电路板：

```
host:~$ sudo uuu -b spl imx-boot
```

您可以像往常一样通过 UART1 console on PEB-EVAL-01 for **phyBOARD-Segin** and X-37 USB-C debug for **phyBOARD-Nash** 在终端上查看启动日志。

备注

使用 UUU 工具启动时，默认的引导命令设置为 fastboot。如果您想更改此设置，请在 U-Boot 提示符下使用 setenv bootcmd_mfg 调整环境变量 bootcmd_mfg。请注意，当使用 UUU 工具启动时，默认环境会被加载。saveenv 没有效果。如果您想永久更改 UUU-boot 的引导命令，则需要在 U-Boot 代码中进行更改。

5.3.5 通过 UUU 工具将 U-boot 镜像烧写到 eMMC

警告

UUU 将 U-boot 刷入 eMMC BOOT（硬件）启动分区后，会在 eMMC 中设置 BOOT_PARTITION_ENABLE。这带来一个问题，因为我们希望 bootloader 驻留在 eMMC USER 分区中。如果刷入下一个 U-Boot 版本的.wic 镜像而不禁用 BOOT_PARTITION_ENABLE 位，将导致设备始终使用保存在 BOOT 分区中的 U-boot。要在 U-Boot 中解决此问题：

```
u-boot=> mmc partconf 0 0 0 0
u-boot=> mmc partconf 0
EXT_CSD[179], PARTITION_CONFIG:
BOOT_ACK: 0x0
BOOT_PARTITION_ENABLE: 0x0
PARTITION_ACCESS: 0x0
```

或从 Linux 检查 Disable booting from eMMC boot partitions。

这样 bootloader 仍然会被刷写到 eMMC 启动分区，但不会被使用！

在使用 **partup** 工具和 `.partup` 包进行 eMMC 烧写时，这一过程是默认进行的，这使得 partup 再次成为优越的烧写选项。

执行并启动电路板：

```
host:~$ sudo uuu -b emmc imx-boot
```

5.3.6 通过 UUU 工具将 wic 镜像刷写到 eMMC

执行并启动电路板：

```
host:~$ sudo uuu -b emmc_all imx-boot phytec-qt6demo-image-phyboard-segin-imx93-2.wic
```

5.4 Standalone Build preparation

在本节中，我们将描述如何在不使用 Yocto Project 的情况下编译 U-Boot 和 Linux 内核。在做他们的开发时可以简化编译 U-Boot 源代码、Linux 内核以及所有其他 git 仓库都可以在我们的 Git 服务器 上找到，地址为 git://git.phytec.de。

备注

如果您的公司防火墙/网关禁止 git 协议，您可以改用 HTTP 或 HTTPS（例如：git clone git://git.phytec.de/u-boot-imx）

5.4.1 Git 仓库

- 使用的 U-Boot 仓库：

```
git://git.phytec.de/u-boot-imx
```

- 我们的 U-Boot 基于 u-boot-imx 并添加了特定于核心板的补丁。
- 使用的 Linux 内核仓库：

```
git://git.phytec.de/linux-imx
```

- 我们的 i.MX 93 内核是基于 linux-imx 内核的。

要找出特定板子应使用的 u-boot 和内核版本 tag，请查看您的 BSP 源文件夹：

```
meta-phytec/dynamic-layers/freescale-layer/recipes-kernel/linux/linux-imx_*.bb
meta-phytec/recipes-bsp/u-boot/u-boot-imx_*.bb
```

5.4.2 获取 SDK

您可以在此处下载 SDK here，或者使用 Yocto 自行构建它：

- 移动到 Yocto 的 build 目录：

```
host:~$ source sources/poky/oe-init-build-env
host:~$ bitbake -c populate_sdk phytec-qt6demo-image # or another image
```

After a successful build the SDK installer is deployed to `build/deploy*/sdk`.

5.4.3 安装 SDK

- 设置正确的权限并安装 SDK：

```
host:~$ chmod +x phytec-ampliphy-vendor-wayland-glibc-x86_64-phytec-qt6demo-image-cortexa55-toolchain-
↪4.2.3.sh
host:~$ ./phytec-ampliphy-vendor-wayland-glibc-x86_64-phytec-qt6demo-image-cortexa55-toolchain-4.2.3.
```

（续下页）

(接上页)

```
↪sh
==========================================================================================
Enter target directory for SDK (default: /opt/ampliphy-vendor-wayland/4.2.3):
You are about to install the SDK to "/opt/ampliphy-vendor-wayland/4.2.3". Proceed [Y/n]? Y
Extracting SDK...done
Setting it up...done
SDK has been successfully set up and is ready to be used.
```

5.4.4 使用 SDK

通过在工具链目录中源文件 *environment-setup* 来为您的 shell 初始化交叉编译工具链：

```
host:~$ source /opt/ampliphy-vendor-wayland/4.2.3/environment-setup-cortexa55-phytec-linux
```

5.4.5 安装所需工具

在树外构建 Linux 和 U-Boot 需要主机安装一些额外的工具依赖项。对于 Ubuntu，您可以使用以下命令安装它们：

```
host:~$ sudo apt install bison flex libssl-dev
```

5.5 U-Boot standalone build

5.5.1 获取源代码

- 获取 U-Boot 源代码：

  ```
  host:~$ git clone git://git.phytec.de/u-boot-imx
  ```

- 要获取正确的 *U-Boot* **tag**，您需要查看我们的 release notes，可以在这里找到：release notes
- 此版本所需的 **tag** 称为 v2023.04_2.2.0-phy3
- 查看所需的 *U-Boot* **tag**：

```
host:~$ cd ~/u-boot-imx/
host:~/u-boot-imx$ git fetch --all --tags
host:~/u-boot-imx$ git checkout tags/v2023.04_2.2.0-phy3
```

- 从技术上讲，您现在可以构建 U-Boot，但实际上还需要一些进一步的步骤：
  - 创建您自己的开发分支：

    ```
    host:~/u-boot-imx$ git switch --create <new-branch>
    ```

    备注

    您可以随意命名您的开发分支，这只是一个示例。

- 设置构建环境：

  ```
  host:~/u-boot-imx$ source /opt/ampliphy-vendor-wayland/4.2.3/environment-setup-cortexa55-phytec-linux
  ```

5.5.2 获取所需的二进制文件

要构建 imx-boot，您需要 **收集**这些 **文件**以便后续使用 **imx-mkimage 工具**：

- **ARM Trusted firmware 二进制文件**（*mkimage* 工具兼容格式 **bl31.bin** ）：bl31-imx93.bin
- **OPTEE 镜像** (可选的)：tee.bin
- **DDR firmware files** (*mkimage* 工具兼容格式 **lpddr4_[i,d]mem_*d_*.bin**): lpddr4_dmem_1d_*.bin, lpddr4_dmem_2d_*.bin, lpddr4_imem_1d_*.bin, lpddr4_imem_2d_*.bin
- **容器镜像**：mx93a1-ahab-container.img

如果您已经使用 Yocto 构建了我们的 BSP，您可以从此处提到的目录中获取这些文件：*BSP Images*

或者您可以从 PHYTEC 下载服务器（https://download.phytec.de/Software/Linux/BSP-Yocto-i.MX93/BSP-Yocto-NXP-i.MX93-PD24.1.1/images/ampliphy-vendor-wayland/phyboard-segin-imx93-2/imx-boot-tools/ ）下载文件。您可以使用下面的命令从该服务器下载所有文件：

```
host:~$ mkdir ./artefacts && cd ./artefacts
host:~/artefacts$ wget \
                    https://download.phytec.de/Software/Linux/BSP-Yocto-i.MX93/BSP-Yocto-NXP-i.MX93-PD24.1.
↪1/images/ampliphy-vendor-wayland/phyboard-segin-imx93-2/imx-boot-tools//bl31-imx93.bin \
                    https://download.phytec.de/Software/Linux/BSP-Yocto-i.MX93/BSP-Yocto-NXP-i.MX93-PD24.1.
↪1/images/ampliphy-vendor-wayland/phyboard-segin-imx93-2/imx-boot-tools//tee.bin \
                    https://download.phytec.de/Software/Linux/BSP-Yocto-i.MX93/BSP-Yocto-NXP-i.MX93-PD24.1.
↪1/images/ampliphy-vendor-wayland/phyboard-segin-imx93-2/imx-boot-tools//lpddr4_dmem_1d_v202201.bin \
                    https://download.phytec.de/Software/Linux/BSP-Yocto-i.MX93/BSP-Yocto-NXP-i.MX93-PD24.1.
↪1/images/ampliphy-vendor-wayland/phyboard-segin-imx93-2/imx-boot-tools//lpddr4_dmem_2d_v202201.bin \
                    https://download.phytec.de/Software/Linux/BSP-Yocto-i.MX93/BSP-Yocto-NXP-i.MX93-PD24.1.
↪1/images/ampliphy-vendor-wayland/phyboard-segin-imx93-2/imx-boot-tools//lpddr4_imem_1d_v202201.bin \
                    https://download.phytec.de/Software/Linux/BSP-Yocto-i.MX93/BSP-Yocto-NXP-i.MX93-PD24.1.
↪1/images/ampliphy-vendor-wayland/phyboard-segin-imx93-2/imx-boot-tools//lpddr4_imem_2d_v202201.bin \
                    https://download.phytec.de/Software/Linux/BSP-Yocto-i.MX93/BSP-Yocto-NXP-i.MX93-PD24.1.
↪1/images/ampliphy-vendor-wayland/phyboard-segin-imx93-2/imx-boot-tools//mx93a1-ahab-container.img
host:~/artefacts$ cd ..
```

5.5.3 构建 bootloader

- 编译 u-boot：

```
host:~/u-boot-imx$ make <defconfig>
host:~/u-boot-imx$ make
host:~/u-boot-imx$ cd ..
```

> **备注**
>
> In command above replace `<defconfig>` with `imx93-phyboard-segin_defconfig` or `imx93-phyboard-nash_defconfig`, depending on the board you are about to build for.

使用 imx-mkimage 编译最终的 flash.bin

- 获取 imx-mkimage：

```
host:~$ git clone https://github.com/nxp-imx/imx-mkimage
host:~$ cd imx-mkimage/
host:~/imx-mkimage$ git checkout tags/lf-6.1.55-2.2.0
```

- 将固件二进制文件复制到 imx-mkimage

```
host:~/imx-mkimage$ cp ../artefacts/bl31-imx93.bin ./iMX9/bl31.bin
host:~/imx-mkimage$ cp \
                      ../artefacts/lpddr4_* \
                      ../artefacts/mx93a1-ahab-container.img \
                      ../artefacts/tee.bin \
                      ./iMX9/
```

- 将 u-boot 二进制文件和 DTB 复制到 imx-mkimage 中

```
host:~/imx-mkimage$ cp ../u-boot-imx/spl/u-boot-spl.bin ../u-boot-imx/u-boot.bin ./iMX9/
host:~/imx-mkimage$ cp ../u-boot-imx/arch/arm/dts/<dtb> ./iMX9/
```

备注

In command above replace `<dtb>` with `imx93-phyboard-segin.dtb` or `imx93-phyboard-nash.dtb`, depending on the board you are about to build for.

- 编译最终的 flash.bin 二进制文件

```
host:~/imx-mkimage$ make SOC=iMX9 REV=A1 flash_singleboot
```

5.5.4 将 bootloader 刷写到块设备上

flash.bin 可以在 imx-mkimage/iMX9/目录下找到，现在可以进行烧写。需要一个特定于芯片的偏移量：

SoC	User 区域偏移量	Boot 分区偏移量	eMMC 设备
i.MX 93	32 kiB	0 kiB	/dev/mmcblk0

例如，烧写 SD 卡：

```
host:~/imx-mkimage$ sudo dd if=./iMX9/flash.bin of=<sd-card> bs=1024 seek=32 conv=sync
```

备注

在上述命令中，将 `<sd-card>` 替换为您的 SD 卡设备名称。有关如何找到设备名称的更多信息，请参阅入门部分中的 Finding the Correct Device 。

提示

具体的偏移值也在 Yocto 变量"BOOTLOADER_SEEK" 和"BOOTLOADER_SEEK_EMMC" 中声明。

5.6 Kernel standalone build

5.6.1 配置源代码

- 使用的 linux-imx 分支可以在 release notes 中找到

- 此版本所需的标签称为 v6.1.55_2.2.0-phy3
- Check out 所需的 linux-imx 标签：

```
host:~$ git clone git://git.phytec.de/linux-imx
host:~$ cd ~/linux-imx/
host:~/linux-imx$ git fetch --all --tags
host:~/linux-imx$ git checkout tags/v6.1.55_2.2.0-phy3
```

- 为了提交更改，强烈建议切换到一个新分支：

```
host:~/linux-imx$ git switch --create <new-branch>
```

- 设置构建环境：

```
host:~/linux-imx$ source /opt/ampliphy-vendor-wayland/4.2.3/environment-setup-cortexa55-phytec-linux
```

5.6.2 编译内核

- 构建 Linux 内核：

```
host:~/linux-imx$ make imx_v8_defconfig imx9_phytec_distro.config imx9_phytec_platform.config
host:~/linux-imx$ make -j$(nproc)
```

- 将内核模块安装到例如 NFS 目录：

```
host:~/linux-imx$ make INSTALL_MOD_PATH=/home/<user>/<rootfspath> modules_install
```

- 镜像可以在 ~/linux-imx/arch/arm64/boot/Image 找到
- dtb 文件可以在 ~/linux-imx/arch/arm64/boot/dts/freescale/imx93-phyboard-segin.dtb 找到
- 要（重新）构建设备树和 -overlay 文件，只需运行

```
host:~/linux-imx$ make dtbs
```

备注

如果您遇到以下构建问题：

```
scripts/dtc/yamltree.c:9:10: fatal error: yaml.h: No such file or directory
```

确保您在主机系统上安装了 *"libyaml-dev"* 包：

```
host:~$ sudo apt install libyaml-dev
```

5.6.3 将内核复制到 SD 卡

当一次性通过网络启动不足以满足需求时，内核及其模块和相应的设备树二进制文件可以直接复制到已挂载的 SD 卡上。

```
host:~/linux-imx$ cp arch/arm64/boot/Image /path/to/sdcard/boot/
host:~/linux-imx$ cp arch/arm64/boot/dts/freescale/imx93-phyboard-segin.dtb /path/to/sdcard/boot/oftree
host:~/linux-imx$ make INSTALL_MOD_PATH=/path/to/sdcard/root/ modules_install
```

5.7 格式化 SD 卡

大多数镜像的大小超过了默认的根分区。要使用 SD 卡进行烧写，根文件系统需要扩展或创建一个单独的分区。有几种不同的方法可以格式化 SD 卡。最简单的方法是使用 UI 程序 Gparted。

5.7.1 Gparted

- 获取 GParted：

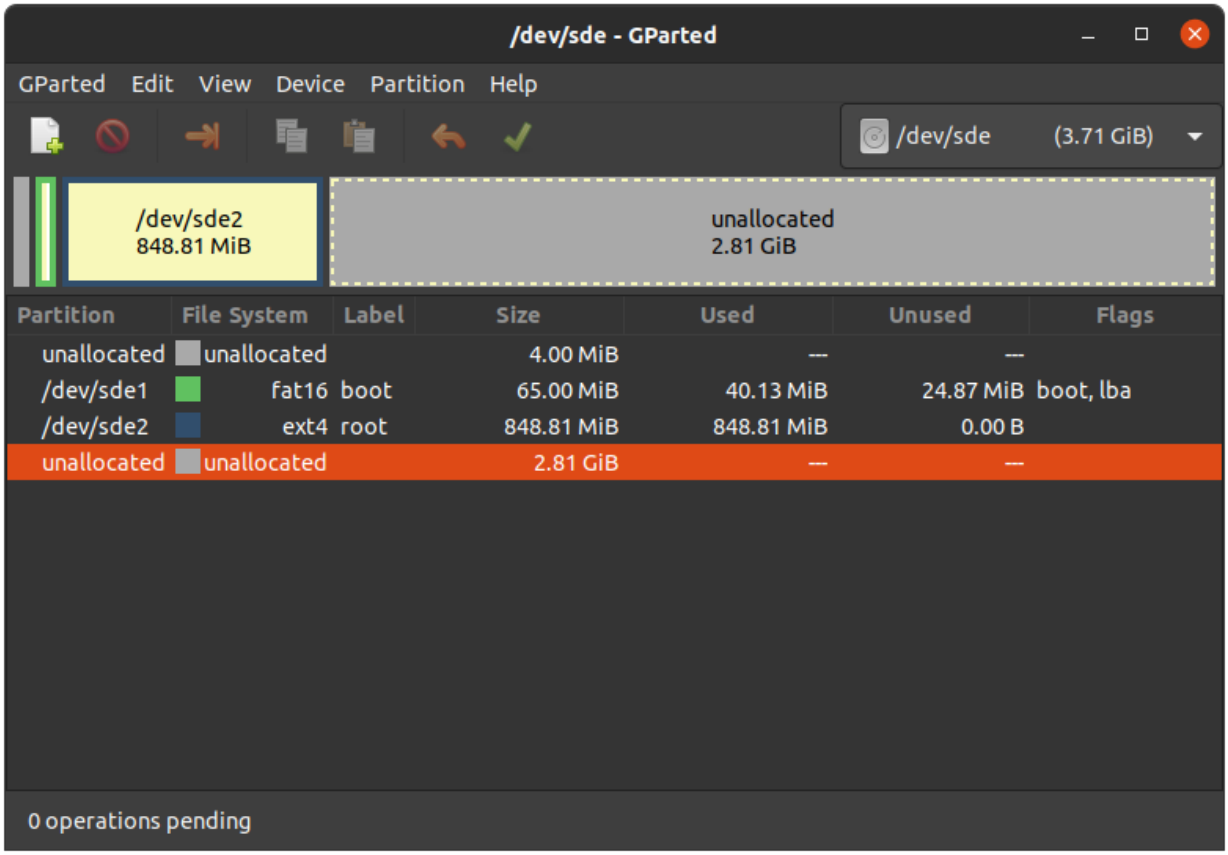
```
host:~$ sudo apt install gparted
```

- 将 SD 卡插入主机并获取设备名称：

```
host:~$ dmesg | tail
...
[30436.175412] sd 4:0:0:0: [sdb] 62453760 512-byte logical blocks: (32.0 GB/29.8 GiB)
[30436.179846]  sdb: sdb1 sdb2
...
```

- 卸载所有 SD 卡分区。
- 启动 GParted：

```
host:~$ sudo gparted
```

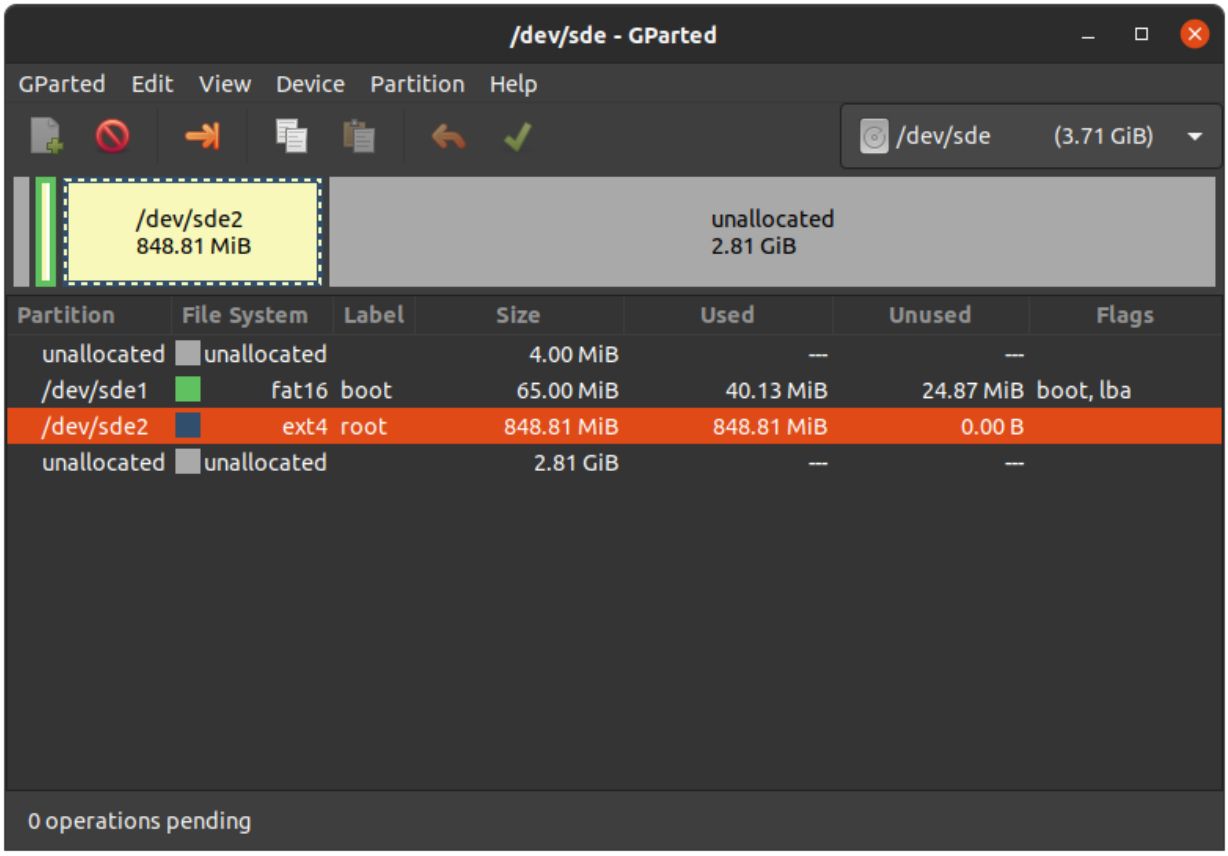


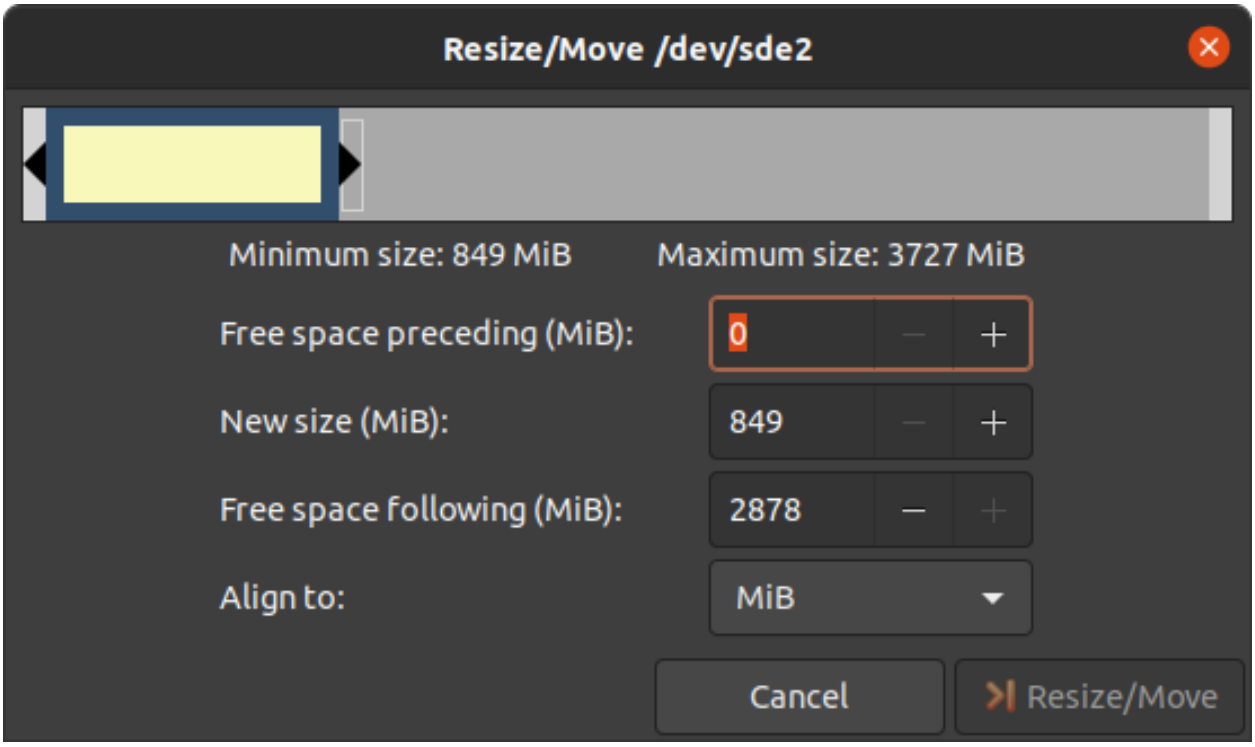
扩展根文件系统

警告

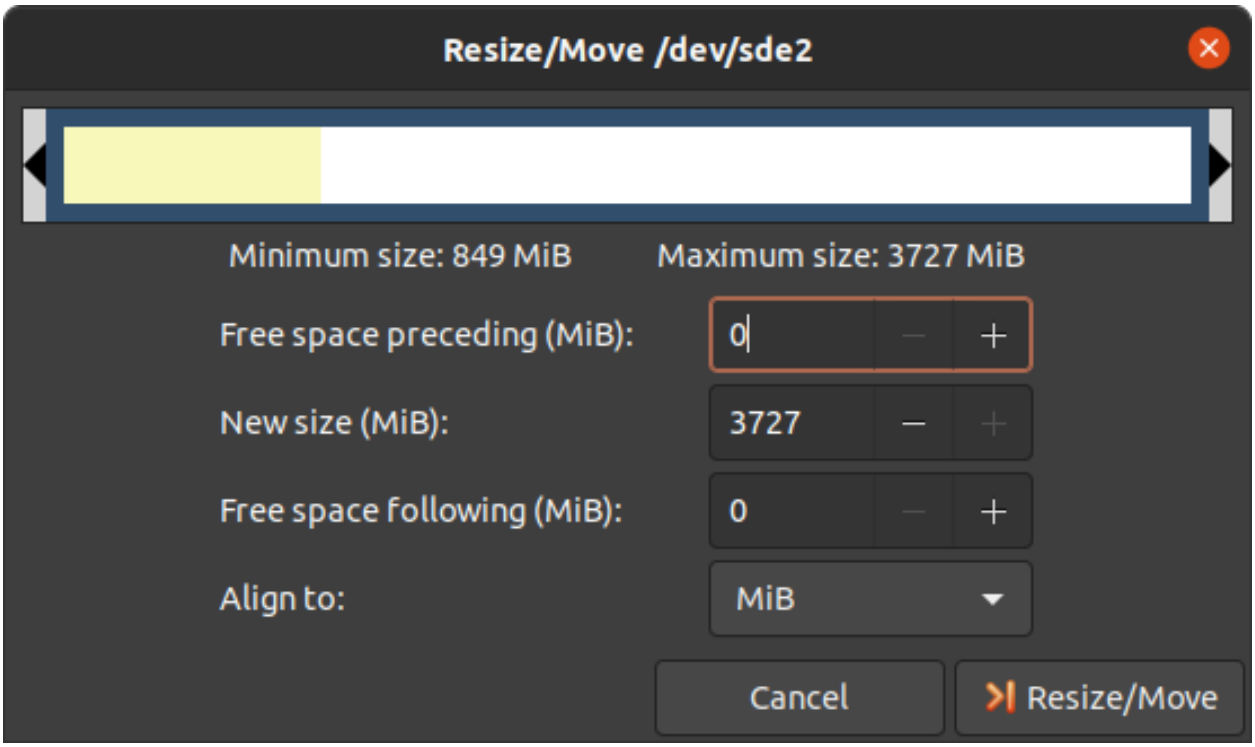
在使用 resize2fs 版本 1.46.6 及更早版本（例如 Ubuntu 22.04）的主机系统上运行 gparted 时，无法扩展使用 Mickledore 版本及更新的 Yocto 创建的 ext4 分区。这是由于 resize2fs 中的一个新默认选项导致的不兼容问题。请参阅 发布说明 。

- 在右上角的下拉菜单中选择您的 SD 卡设备
- 选择 ext4 根分区并点击调整大小：

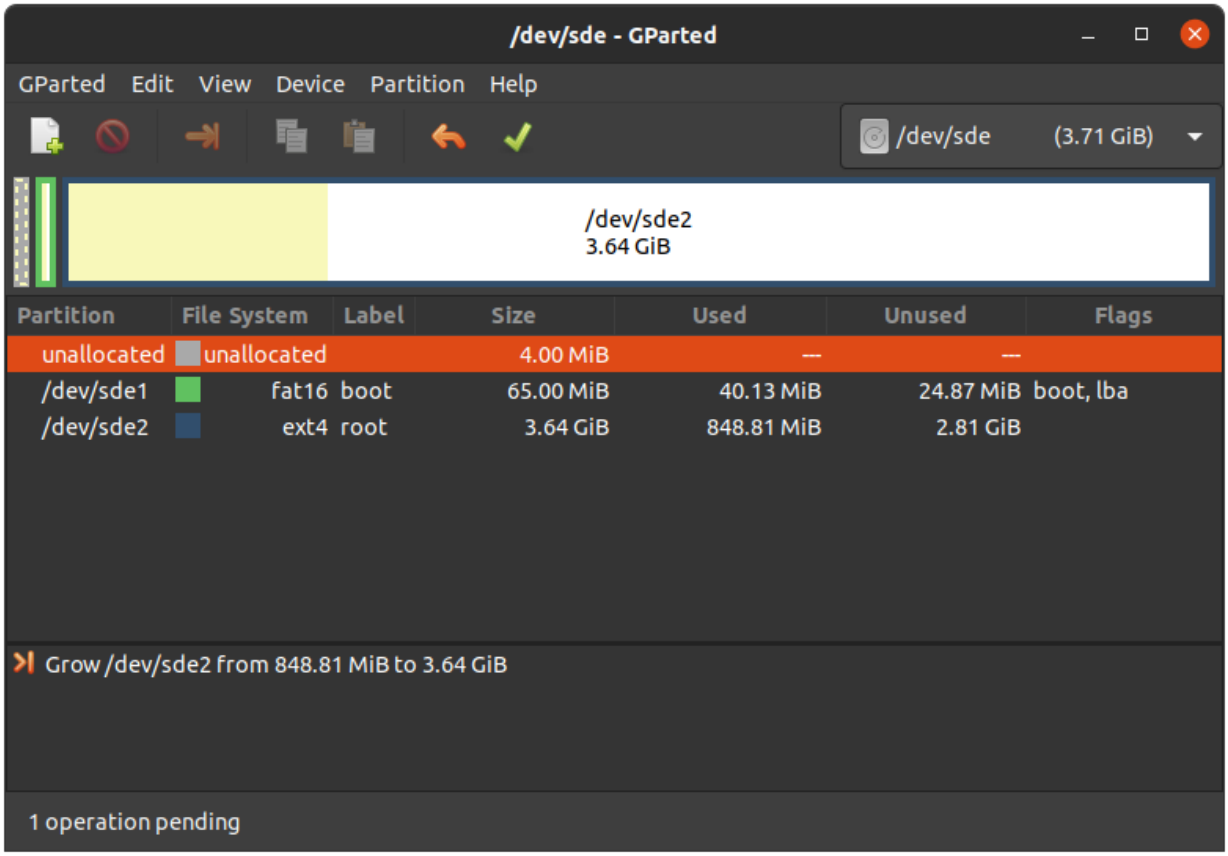




- 您可以根据需要拖动滑块或手动输入大小。



- 通过点击“更改大小”按钮确认您的输入。

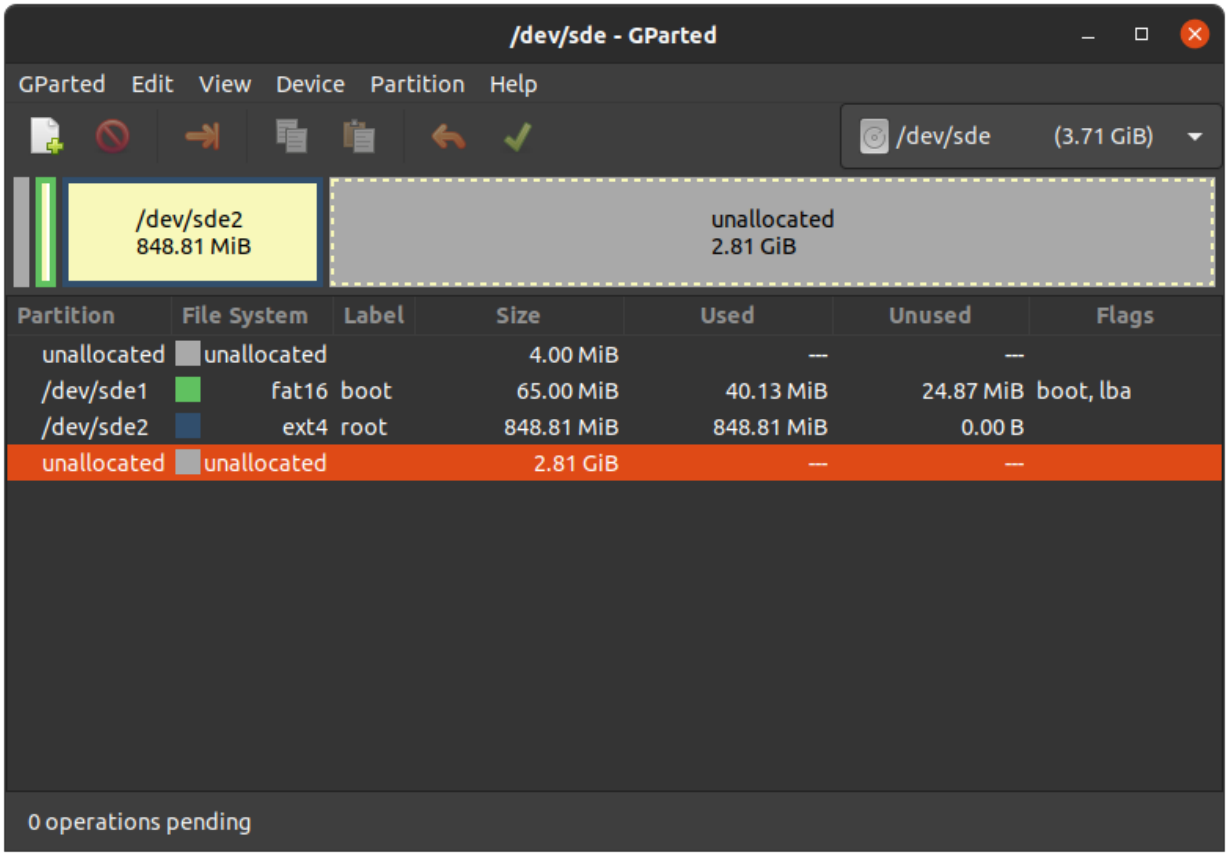


- 要应用您的更改，请按绿色勾号。
- 现在您可以挂载根分区并将例如 phytec-qt6demo-image-phyboard-segin-imx93-2.wic 镜像复制到其中。然后再卸载它：

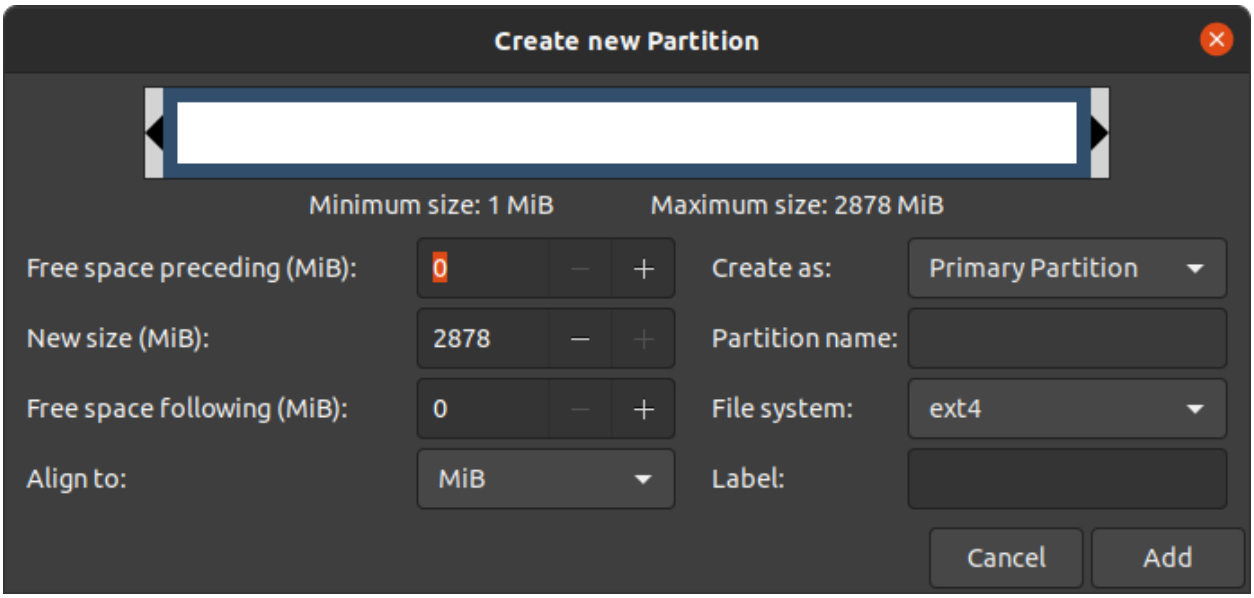
```
host:~$ sudo cp phytec-qt6demo-image-phyboard-segin-imx93-2.wic /mnt/ ; sync
host:~$ umount /mnt
```

创建第三个分区

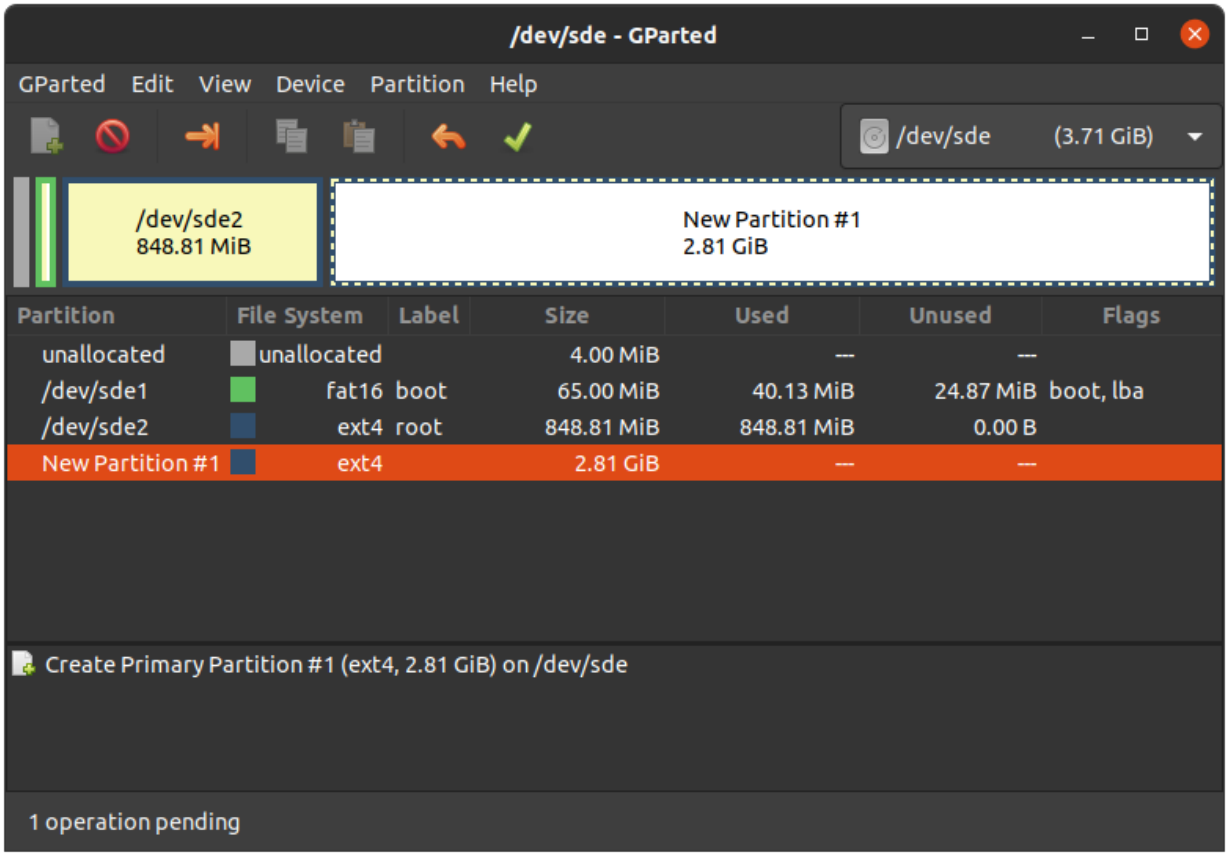
- 在右上角的下拉菜单中选择您的 SD 卡设备



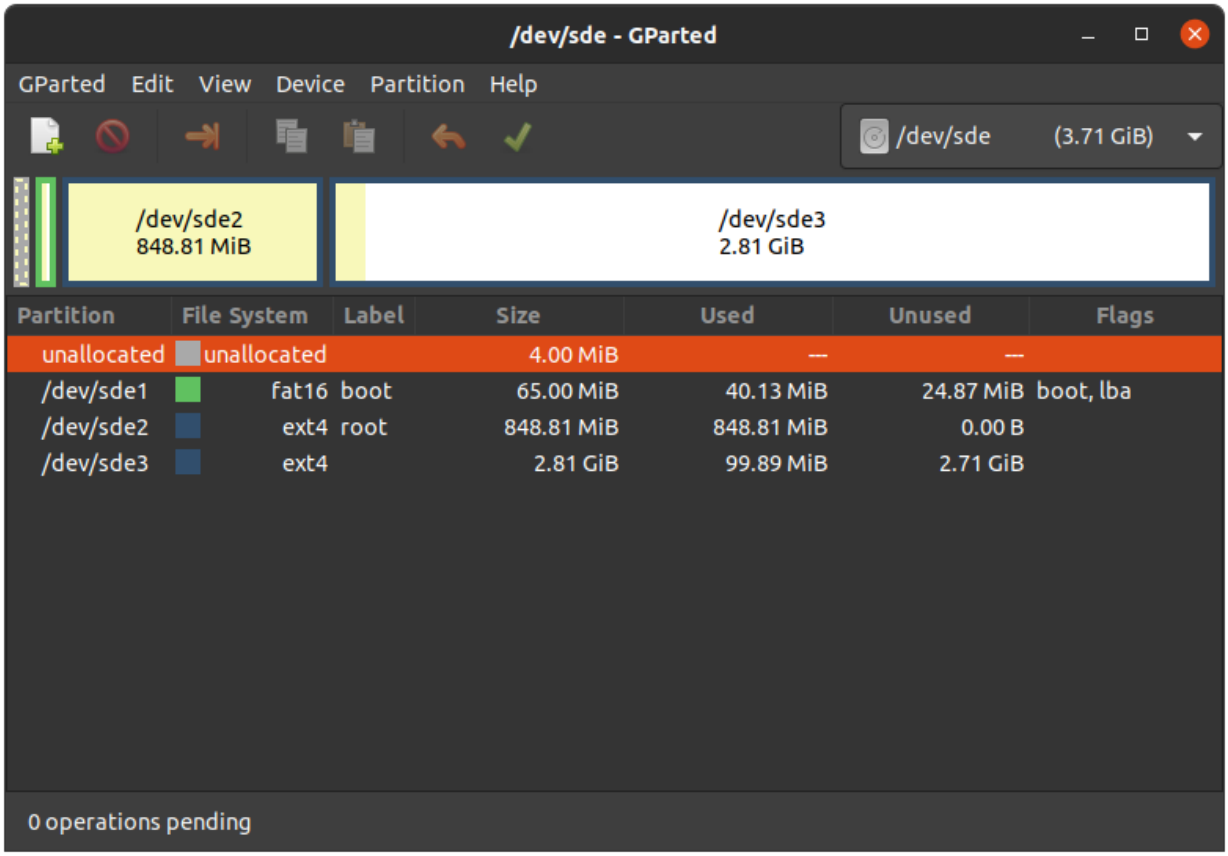
- 选择更大的未分配区域，然后点击"New"：



- 点击"Add"



- 通过按绿色勾选确认您的更改。



- 现在您可以挂载新的分区并将例如 phytec-qt6demo-image-phyboard-segin-imx93-2.wic 镜像复制到其中。然后卸载它：

```
host:~$ sudo mount /dev/sde3 /mnt
host:~$ sudo cp phytec-qt6demo-image-phyboard-segin-imx93-2.wic /mnt/ ; sync
host:~$ umount /mnt
```

CHAPTER 6

设备树 (DT)

6.1 介绍

以下文本简要描述了设备树，关于设备树的相关文档可以在 Linux 内核文档中找到（https://docs.kernel.org/devicetree/usage-model.html）。

"Open Firmware Device Tree" 或简称设备树（DT）是一种用于描述硬件的数据结构和语言。更具体地说，它是一个可由操作系统读取的硬件描述，以便操作系统不需要对机器的细节进行硬编码

内核文档是设备树介绍的一个非常好的资源。关于设备树数据格式的概述可以在 devicetree.org 的设备树使用页面找到。

6.2 PHYTEC i.MX 93 BSP 设备树概念

以下部分解释了 PHYTEC 在如何为我们的基于 SoC i.MX 93 的核心板设备树所定义的一些规则。

6.2.1 设备树结构

- *Module.dtsi* - 模块包括所有安装在核心板上的设备，例如 PMIC 和 RAM。
- *Board.dts* - 包含核心板 dtsi 文件。底板使用的来自 SoC i.MX 93 的设备也包含在此 dts 中。
- *Overlay.dtso* - 根据安装或未连接在核心板或底板上的可选硬件（例如 SPI 闪存或 PEB-AV-10）的情况，启用/禁用功能。

从 Linux 内核的根目录，我们的 i.MX 9 平台的设备树文件可以在 `arch/arm64/boot/dts/freescale/` 找到。

6.2.2 设备树 Overlay

Device Tree overlays are device tree fragments that can be merged into a device tree during boot time. These are for example hardware descriptions of an expansion board. They are instead of being added to the device tree as an extra include, now applied as an overlay. They also may only contain setting the status of a node depending on if it is mounted or not. The device tree overlays are placed next to the other device tree files in our Linux kernel repository in the folder `arch/arm64/boot/dts/freescale/`.

可用的 overlay 文件为 phyboard-segin-imx93-2.conf 有：

```
imx93-phyboard-segin-peb-av-02.dtbo
imx93-phyboard-segin-peb-eval-01.dtbo
imx93-phycore-rpmsg.dtbo
imx93-phycore-no-emmc.dtbo
imx93-phycore-no-eth.dtbo
```

phyboard-nash-imx93-1.conf 可用的 overlay 有：

```
imx93-phyboard-nash-peb-av-010.dtbo
imx93-phycore-rpmsg.dtbo
imx93-phycore-no-emmc.dtbo
imx93-phycore-no-eth.dtbo
```

可以在 Linux 或 U-Boot 运行时配置 overlay。overlay 是在引导命令调用后、内核加载之前的引导过程中生效。接下来的部分将更详细地解释配置方法。

设置 ${overlays} 变量

`${overlays}` U-Boot 环境变量包含一个以空格分隔的 overlay 文件列表，这些 overlay 文件将在启动过程中应用。根据启动源，overlay 文件必须放置在 eMMC/SD 卡的启动分区中，或者通过 tftp 加载。在 $KERNEL_DEVICETREE 这个 Yocto machine 变量中设置的 overlay 文件将自动添加到最终 WIC 镜像的启动分区中。

`${overlays}` 变量可以直接在 U-Boot 环境中设置，也可以作为外部 `bootenv.txt` 环境文件的一部分。默认情况下，`${overlays}` 变量来自位于启动分区的外部 `bootenv.txt` 环境文件。您可以在启动的目标设备上从 Linux 读取和写入该文件：

```
target:~$ cat /boot/bootenv.txt
overlays=imx93-phyboard-segin-peb-eval-01.dtbo imx93-phyboard-segin-peb-av-02.dtbo
```

备注

确保 boot 分区已挂载！如果没有，您可以使用以下命令进行挂载：

```
target:~$ mount /dev/mmcblkXp0 /boot
```

更改将在下次重启后生效。如果没有可用的 `bootenv.txt` 文件，则可以直接在 U-Boot 环境中设置 overlay 变量。

```
u-boot=> setenv overlays imx93-phyboard-segin-peb-av-02.dtbo
u-boot=> printenv overlays
overlays=imx93-phyboard-segin-peb-av-02.dtbo
u-boot=> boot
```

如果用户定义的 `${overlays}` 变量应该直接从 U-Boot 环境加载，但仍然有一个外部的 `bootenv.txt` 文件，则需要将设置 `${no_bootenv}` 变量：

```
u-boot=> setenv no_bootenv 1
u-boot=> setenv overlays imx93-phyboard-segin-peb-av-02.dtbo
u-boot=> boot
```

有关外部环境的更多信息，请参见 U-boot External Environment subsection of the *device tree overlay section*。

我们使用 `${overlays}` 变量来描述在运行时无法检测到的扩展板。为了防止在启动时应用列在 `${overlays}` 变量中的 overlay，可以在 bootloader 环境中将 `${no_overlays}` 变量设置为 *1*。

```
u-boot=> setenv no_overlays 1
u-boot=> boot
```

6.2.3 U-boot 外部环境

在 Linux 内核启动时，外部环境 `bootenv.txt` 文本文件将从 MMC 设备的引导分区或通过 TFTP 加载。该文件的主要目的是存储 `${overlays}` 变量。这使得根据使用的机器在 Yocto 中预定义 overlay 变得更加容易。文件的内容在 meta-phytec 中的 Yocto recipe 中的 bootenv 中定义：https://git.phytec.de/meta-phytec/tree/recipes-bsp/bootenv?h=mickledore

该文件中也可以设置其他变量。这些变量将覆盖环境中现有的设置。但只有在发出 boot 命令后计算的变量才能被覆盖，例如 `${nfsroot}` 或 `${mmcargs}`。更改在该文件修改其他变量将不会有作用。请参见网络启动期间的用法作为示例。

如果无法加载外部环境，启动过程将继续进行，并使用现有环境设置的值。

6.2.4 从 Linux 更改目标上的 U-boot 环境

Libubootenv 是我们镜像中包含的一个工具，用于在目标机器 linux 上修改 U-Boot 环境。

使用以下命令打印 U-Boot 环境：

```
target:~$ fw_printenv
```

使用以下命令修改 U-Boot 环境变量：

```
target:~$ fw_setenv <variable> <value>
```

小心

Libubootenv 使用在配置文件中选择的环境。要使用的环境会被插入到该文件中，默认情况下它被配置为使用 eMMC 环境（即默认使用的环境）。

如果 eMMC 没有被烧写过或者 eMMC 环境被删除，libubootenv 将无法工作。您应该修改 `/etc/fw_env.config` 文件，以匹配您想要使用的环境源。

CHAPTER 7

访问外设

要查找本文中所述的 PHYTEC 的 phyCORE-i.MX 93 BSP 支持的板卡和核心板，请访问 our BSP 网页，并在下载部分点击相应的 BSP 版本。在这里，您可以在”硬件订货号”列中找到所有支持的硬件，并在“Machine Name”下的相应单元格中找到正确的机器名称。

为了实现最大的软件重用，Linux 内核提供了一个巧妙的软件架构，将软件组件分层为特定于板的部分。BSP（板级支持包）尽可能地对套件的功能进行模块化。当开发定制的主板或甚至客户特定的模块时，大部分软件支持可以在不易出错的复制粘贴情况下重新使用。与板相关的内核代码可以在内核代码库中的设备树（DT）中找到，路径为 `arch/arm64/boot/dts/freescale/*.dts` 。

实际上，软件重用是 Linux 内核最重要的特性之一，尤其是在 ARM 实现中，它必须应对大量复杂的系统级芯片（SoC）CPU 的可能性。整个特定于板的硬件在设备树（DT）中描述，并不属于内核镜像本身。硬件描述在一个单独的二进制文件中，称为设备树二进制文件（Device Tree Blob，DTB）（参见*device tree*）。

请阅读 PHYTEC i.MX 93 BSP 设备树概念部分，以了解我们的 i.MX 9 BSP 设备树模型。

以下部分提供了 i.MX 9 平台上支持的硬件组件及其操作系统驱动程序的概述。进一步的更改可以根据客户的要求进行移植。

7.1 i.MX 93 引脚复用

该 i.MX 93 系统芯片包含许多外设接口。为了在保持最大功能的同时减少封装尺寸和降低整体系统成本，许多 i.MX 93 引脚可以多路复用多达八种信号功能。尽管存在许多可能的引脚多路复用组合，但由于时序限制，只有一定数量的组合被称为有效的 IO 集合。这些有效的 IO 集合经过精心选择，以为用户提供多种可能的应用场景。

请参考我们的硬件手册或 NXP i.MX 93 参考手册，以获取有关特定引脚和复用能力的更多信息。

IO 设置配置，也称为复用（muxing），是在设备树中完成的。驱动程序 pinctrl-single 读取设备树的节点 fsl,pins，并进行适当的引脚复用。

以下是 imx93-phyboard-segin.dts 中 UART1 设备的引脚复用示例：

```
pinctrl_uart1: uart1grp {
        fsl,pins = <
```

(续下页)

(接上页)

```
                MX93_PAD_UART1_RXD__LPUART1_RX     0x31e
                MX93_PAD_UART1_TXD__LPUART1_TX     0x30e
        >;
};
```

字符串的第一部分 MX93_PAD_UART1_RXD__LPUART1_RX 指定了引脚（在这个例子中是 UART1_RXD）。字符串的第二部分（LPUART1_RX）是该引脚的期望复用选项。引脚设置值（右侧的十六进制值）定义了引脚的不同模式，例如，内部上拉电阻是否被激活。在当前情况下，内部上拉电阻被激活。

UART1 引脚复用的设备树表示：https://git.phytec.de/linux-imx/tree/arch/arm64/boot/dts/freescale/imx93-phyboard-segin.dts?h=v6.1.55_2.2.0-phy3#n262

7.2 网络

phyBOARD-Segin/Nash i.MX 93 提供两个以太网接口。

- 在 **phyBOARD-Segin** 上，我们有：
  - 由 phyCORE-i.MX93 提供的百兆以太网
  - 由 phyBOARD 提供的百兆以太网。
- 在 **phyBOARD-Nash** 上，我们有：
  - 由 phyCORE-i.MX93 提供的百兆以太网
  - 由 phyBOARD 提供的千兆以太网。

所有接口都提供一个标准的 Linux 网络端口，可以使用 BSD 套接字接口进行编程。整个网络配置由 systemd-networkd 守护进程处理。相关的配置文件可以在目标设备的 `/lib/systemd/network/` 目录中找到，以及在 BSP 中的 `meta-ampliphy/recipes-core/systemd/systemd-conf` 目录中。

IP 地址可以在 *.network 文件中进行配置。eth0 的默认 IP 地址和子网掩码为：

```
eth0: 192.168.3.11/24
```

根据您的硬件配置，设备树的以太网设置可能会分为两个文件：核心板的 DT 文件和底板的 DT。FEC 以太网 IP 核心的设备树设置，其中以太网 PHY 被集成在核心板上，可以在这里找到：https://git.phytec.de/linux-imx/tree/arch/arm64/boot/dts/freescale/imx93-phycore-som.dtsi?h=v6.1.55_2.2.0-phy3#n61。

The device tree set up for EQOS Ethernet IP core where the PHY is populated on the phyBOARD-Segin/Nash i.MX 93 can be found here: https://git.phytec.de/linux-imx/tree/arch/arm64/boot/dts/freescale/imx93-phyboard-segin.dts?h=v6.1.55_2.2.0-phy3#n114 or here: https://git.phytec.de/linux-imx/tree/arch/arm64/boot/dts/freescale/imx93-phyboard-nash.dts?h=v6.1.55_2.2.0-phy3#n83.

7.2.1 网络环境定制化

U-boot 网络环境

- 要在 U-Boot 中查找以太网设置：

  ```
  u-boot=> printenv ipaddr serverip netmask
  ```

- 在将开发主机设置为 IP 192.168.3.10 和子网掩码 255.255.255.0 的情况下，目标应该返回：

```
u-boot=> printenv ipaddr serverip netmask
ipaddr=192.168.3.11
serverip=192.168.3.10
netmask=255.225.255.0
```

- 如果您需要进行任何更改：

```
u-boot=> setenv <parameter> <value>
```

  <parameter> 应该是 ipaddr、netmask、gatewayip 或 serverip 中的一个。<value> 将是所选参数的实际值。

- 您所做的更改目前是临时的。要保存这些更改：

```
u-boot=> saveenv
```

在这里，您也可以将 IP 地址更改为 DHCP，而不是使用静态 IP 地址。

- 配置：

```
u-boot=> setenv ip dhcp
```

- 设置 TFTP 和 NFS 的路径。修改可以如下所示：

```
u-boot=> setenv nfsroot /home/user/nfssrc
```

请注意，这些修改只会影响 bootloader 设置。

小技巧

像 nfsroot 和 netargs 这样的变量可以被 U-boot 外部环境覆盖。对于网络启动，外部环境将通过 tftp 加载。例如，要在 `bootenv.txt` 文件中本地设置 nfsroot 变量，请找到 tftproot 目录：

```
nfsroot=/home/user/nfssrc
```

无需在目标上存储信息。请注意，这对于像 ipaddr 或 serveraddr 这样的变量不起作用，因为在加载外部环境时，它们需要已经被设置好。

内核网络环境

- 在目标内核中查找 eth0 的以太网设置：

```
target:~$ ifconfig eth0
eth0      Link encap:Ethernet  HWaddr 50:2D:F4:19:D6:33
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
```

- eth0 配置的临时调整：

```
target:~$ ifconfig eth0 192.168.3.11 netmask 255.255.255.0 up
```

7.3 SD/MMC 卡

该 i.MX 93 支持一个插槽，用于 SD 卡和 MMC 卡，作为 linux 通用块设备。这些设备可以像其他任何块设备一样使用。

警告

这些设备是热插拔的。然而，您必须确保在设备仍然挂载时不要拔掉它。这可能会导致数据丢失！

插入 SD/MMC 卡后，内核将在/dev 中生成新的设备节点。完整设备可以通过其/dev/mmcblk1 设备节点访问。SD/MMC 卡的分区将显示为：

```
/dev/mmcblk1p<Y>
```

<Y> 作为分区编号，从 1 开始计数，直到该设备的最大分区数量。分区可以使用任何类型的文件系统进行格式化，并且可以以标准方式进行处理，例如，mount 和 umount 命令按预期工作。

小技巧

这些分区设备节点仅在 SD 卡包含有效的分区表时可用（类似于“硬盘”的处理）。如果没有分区表，则整个设备可以作为文件系统使用（类似于“软盘”的处理）。在这种情况下，必须使用 /dev/mmcblk1 进行格式化和挂载。卡片始终以可写方式挂载。

MMC（SD 卡插槽）接口的设备树配置可以在这里找到：https://git.phytec.de/linux-imx/tree/arch/arm64/boot/dts/freescale/imx93-phyboard-segin.dts?h=v6.1.55_2.2.0-phy3#n216 或者这里：https://git.phytec.de/linux-imx/tree/arch/arm64/boot/dts/freescale/imx93-phyboard-nash.dts?h=v6.1.55_2.2.0-phy3#n201

eMMC 接口的 DT 配置可以在这里找到：https://git.phytec.de/linux-imx/tree/arch/arm64/boot/dts/freescale/imx93-phycore-som.dtsi?h=v6.1.55_2.2.0-phy3#n194 或者在这里：

7.4 eMMC 设备

PHYTEC 模块如 phyCORE-i.MX 93 配备了 eMMC 存储芯片作为主要存储。eMMC 设备使用多层单元（MLC）或三层单元（TLC）技术来实现存储，并结合了处理 ECC 和磨损均衡的存储控制器。它们通过 SD/MMC 接口连接到 i.MX 93 ，并在 Linux 内核中作为块设备表示，如 SD 卡、优盘或硬盘。

电气和协议规范由 JEDEC 提供（https://www.jedec.org/standards-documents/technology-focus-areas/flash-memory-ssds-ufs-emmc/e-mmc）。eMMC 制造商的数据手册相对较短，旨在与支持的 JEDEC eMMC 标准版本一起阅读。

PHYTEC 目前使用 JEDEC 版本 5.0 和 5.1 的 eMMC 芯片。

7.4.1 扩展 CSD 寄存器

eMMC 设备具有大量额外的信息和设置，这些信息和设置可以通过扩展 CSD 寄存器获得。有关寄存器的详细列表，请参阅制造商的数据手册和 JEDEC 标准。

在 Linux 用户空间中，您可以查询寄存器：

```
target:~$ mmc extcsd read /dev/mmcblk0
```

你将会看到：

```
===============================================
   Extended CSD rev 1.7 (MMC 5.0)
===============================================

 Card Supported Command sets [S_CMD_SET: 0x01]
 [...]
```

7.4.2 启用后台操作 (BKOPS)

与原始 NAND Flash 相比，eMMC 设备包含一个闪存传输层（FTL），该层负责处理原始 MLC 或 TLC 的磨损均衡、块管理和错误更正码（ECC）。这需要定期执行一些维护任务（例如擦除未使用的块）。这些任务被称为 **后台操作（BKOPS）**。

默认情况下（取决于芯片），后台操作可能会定期执行，也可能不会，这会影响最坏情况下的读写延迟。

JEDEC 标准自版本 v4.41 起规定了一种方法，主机可以手动发出 BKOPS。有关更多详细信息，请参阅 JEDEC 标准章节“背景操作”以及 eMMC 数据手册中寄存器 BKOPS_EN（寄存器：163）和 BKOPS_START（寄存器：164）的描述。

寄存器 BKOPS_EN（寄存器：163）的位 MANUAL_EN（位 0）的含义：

- 值 0：主机不支持手动触发 BKOPS。设备写入性能受到影响。
- 值 1：主机支持手动触发 BKOPS。当主机不需要设备时，它会不时发出 BKOPS。

自 v3.7 版本以来，Linux 内核已经实现了发起后台操作的机制。您只需在 eMMC 设备上启用 BKOPS_EN（详细信息见下文）。

JEDEC 标准 v5.1 引入了一种新的自动 BKOPS 功能。它使主机能够定期触发后台操作，因为设备在空闲时会自动启动 BKOPS（请参见寄存器 BKOPS_EN（寄存器：163）中位 AUTO_EN 的描述）。

- 要检查 *BKOPS_EN* 是否已设置，请执行：

  ```
  target:~$ mmc extcsd read /dev/mmcblk0 | grep BKOPS_EN
  ```

  输出将会是，例如：

  ```
  Enable background operations handshake [BKOPS_EN]: 0x01
  #OR
  Enable background operations handshake [BKOPS_EN]: 0x00
  ```

  值 0x00 表示 BKOPS_EN 被禁用，设备的写入性能受到影响。值 0x01 表示 BKOPS_EN 被启用，主机将不时发起后台操作。

- 启用可以通过以下命令完成：

  ```
  target:~$ target:~$ mmc --help

  [...]
  mmc bkops_en <auto|manual> <device>
      Enable the eMMC BKOPS feature on <device>.
      The auto (AUTO_EN) setting is only supported on eMMC 5.0 or newer.
      Setting auto won't have any effect if manual is set.
      NOTE!  Setting manual (MANUAL_EN) is one-time programmable (unreversible) change.
  ```

- 要设置 BKOPS_EN 位，请执行：

  ```
  target:~$ mmc bkops_en manual /dev/mmcblk0
  ```

- 为了确保新设置生效并且内核能够自动触发 BKOPS，请关闭系统：

```
target:~$ poweroff
```

小技巧

BKOPS_EN 位是一次性可编程的，无法恢复。

7.4.3 可靠写入

有两种不同的可靠写入选项：

1. 整个 eMMC 设备/分区的可靠写入选项。
2. 单次写事务的可靠写入。

小技巧

不要将 eMMC 分区与 DOS、MBR 或 GPT 分区表的分区混淆（请参阅前一节）。

第一个可靠写入选项在 phyCORE-i.MX 93 SoM 上挂载的 eMMC 上大多数情况下已经启用。要在运行的目标上检查这一点：

```
target:~$ mmc extcsd read /dev/mmcblk0 | grep -A 5 WR_REL_SET
Write reliability setting register [WR_REL_SET]: 0x1f
 user area: the device protects existing data if a power failure occurs during a write o
peration
 partition 1: the device protects existing data if a power failure occurs during a write
 operation
 partition 2: the device protects existing data if a power failure occurs during a write
 operation
 partition 3: the device protects existing data if a power failure occurs during a write
 operation
 partition 4: the device protects existing data if a power failure occurs during a write
 operation
--
 Device supports writing EXT_CSD_WR_REL_SET
 Device supports the enhanced def. of reliable write
```

否则，可以使用 mmc 工具启用它：

```
target:~$ mmc --help

[...]
mmc write_reliability set <-y|-n|-c> <partition> <device>
    Enable write reliability per partition for the <device>.
    Dry-run only unless -y or -c is passed.
    Use -c if more partitioning settings are still to come.
    NOTE!  This is a one-time programmable (unreversible) change.
```

第二个可靠写入选项是命令 CMD23 中的配置位 Reliable Write Request parameter（可靠写入请求参数）（位 31）。自内核版本 v3.0 起，文件系统（例如 ext4 的日志）和用户空间应用程序（如 fdisk 的分区表）会通过内核使用该可靠写入功能。在 Linux 内核源代码中，它通过标志 REQ_META 进行处理。

结论：使用挂载选项 data=journal 的 ext4 文件系统在断电情况下应该是安全的。文件系统检查可以在断电后恢复文件系统，但在断电前刚写入的数据可能会丢失。无论如何，可以恢复文件系统的正常状态。为了确保应用程序文件的数据正常保存，应用程序中应使用系统函数 fdatasync 或 fsync。

7.4.4 调整 ext4 根文件系统的大小

在将 SD 卡镜像写入 eMMC 时，ext4 根分区没有扩展到 eMMC 的末尾。可以使用 parted 来扩展根分区。这个示例适用于任何块设备，例如 eMMC、SD 卡或硬盘。

- 获取当前设备大小：

```
target:~$ parted /dev/mmcblk0 print
```

- 输出如下：

```
Model: MMC Q2J55L (sd/mmc)
Disk /dev/mmcblk0: 7617MB
Sect[ 1799.850385]  mmcblk0: p1 p2
or size (logical/physical): 512B/512B
Partition Table: msdos
Disk Flags:

Number  Start   End     Size    Type     File system  Flags
 1      4194kB  72.4MB  68.2MB  primary  fat16        boot, lba
 2      72.4MB  537MB   465MB   primary  ext4
```

- 使用 parted 将根分区调整为设备的最大大小：

```
target:~$ parted /dev/mmcblk0 resizepart 2 100%
Information: You may need to update /etc/fstab.

target:~$ parted /dev/mmcblk0 print
Model: MMC Q2J55L (sd/mmc)
Disk /dev/mmcblk0: 7617MB
Sector size (logical/physical): 512[ 1974.191657]  mmcblk0: p1 p2
B/512B
Partition Table: msdos
Disk Flags:

Number  Start   End     Size    Type     File system  Flags
 1      4194kB  72.4MB  68.2MB  primary  fat16        boot, lba
 2      72.4MB  7617MB  7545MB  primary  ext4
```

在文件系统挂载时可以增加其大小。但您也可以从 SD 卡启动板，然后在 eMMC 分区未挂载时调整文件系统的大小。

- 将文件系统调整为新的分区大小：

```
target:~$ resize2fs /dev/mmcblk0p2
resize2fs 1.46.1 (9-Feb-2021)
Filesystem at /dev/mmcblk0p2 is mounted on /; on-line resizing required
[ 131.609512] EXT4-fs (mmcblk0p2): resizing filesystem
from 454136 to 7367680 blocks
old_desc_blocks = 4, new_desc_blocks = 57
[ 131.970278] EXT4-fs (mmcblk0p2): resized filesystem to 7367680
The filesystem on /dev/mmcblk0p2 is now 7367680 (1k) blocks long
```

7.4.5 启用伪 SLC 模式

eMMC 设备使用 MLC 或 TLC 来存储数据。与 NAND Flash 中使用的 SLC 相比，MLC 或 TLC 在成本更低的情况下，可靠性较低且错误率较高。

如果您更喜欢可靠性而不是存储容量，可以启用伪 SLC 模式或 SLC 模式。这里使用的方法采用了增强属性，该属性在 JEDEC 标准中有所描述，可以为设备的连续区域设置。JEDEC 标准并未规定增强属性的实现细节和保障，这由芯片制造商自行决定。对于美光（Micron）芯片，增强属性提高了可靠性，但也将容量减半。

警告

在设备上启用增强属性时，所有数据将被丢失。

以下步骤展示了如何启用增强属性。

- 首先使用以下命令获取 eMMC 设备的当前大小：

```
target:~$ parted -m /dev/mmcblk0 unit B print
```

你将收到：

```
BYT;
/dev/mmcblk0:63652757504B:sd/mmc:512:512:unknown:MMC S0J58X:;
```

如您所见，该设备的容量为 63652757504 字节 = 60704 MiB。

- 要在启用伪 SLC 后获取设备的最大大小，请使用：

```
target:~$ mmc extcsd read /dev/mmcblk0 | grep ENH_SIZE_MULT -A 1
```

例如：

```
Max Enhanced Area Size [MAX_ENH_SIZE_MULT]: 0x000764
i.e. 3719168 KiB
--
Enhanced User Data Area Size [ENH_SIZE_MULT]: 0x000000
i.e. 0 KiB
```

这里的最大大小是 3719168 KiB = 3632 MiB。

- 现在，您可以通过输入以下命令为整个设备设置增强属性，例如 3719168 KiB：

```
target:~$ mmc enh_area set -y 0 3719168 /dev/mmcblk0
```

你将获得：

```
Done setting ENH_USR area on /dev/mmcblk0
setting OTP PARTITION_SETTING_COMPLETED!
Setting OTP PARTITION_SETTING_COMPLETED on /dev/mmcblk0 SUCCESS
Device power cycle needed for settings to take effect.
Confirm that PARTITION_SETTING_COMPLETED bit is set using 'extcsd read' after power cycle
```

- 为了确保新设置已生效，请关闭系统：

```
target:~$ poweroff
```

并进行完整的上下电。建议您确认设置。

- 首先，检查 ENH_SIZE_MULT 的值，它必须是 3719168 KiB：

```
target:~$ mmc extcsd read /dev/mmcblk0 | grep ENH_SIZE_MULT  -A 1
```

您应该看到：

```
Max Enhanced Area Size [MAX_ENH_SIZE_MULT]: 0x000764
i.e. 3719168 KiB
--
Enhanced User Data Area Size [ENH_SIZE_MULT]: 0x000764
i.e. 3719168 KiB
```

- 最后，检查设备的大小：

```
target:~$ parted -m /dev/mmcblk0 unit B print
BYT;
/dev/mmcblk0:31742492672B:sd/mmc:512:512:unknown:MMC S0J58X:;
```

7.4.6 擦除设备

可以直接擦除 eMMC 设备，而不是通过写零覆盖它。eMMC 块管理算法将擦除底层的 MLC 或 TLC，或者将这些块标记为可丢弃。设备上的数据将丢失，并将被读取为零。

- 从 SD 卡启动后执行：

```
target:~$ blkdiscard -f --secure /dev/mmcblk0
```

  选项 --secure 确保命令在 eMMC 设备擦除所有块之前会等待。-f (强制) 选项在擦除之前禁用所有检查，当 eMMC 设备包含已有数据的分区时需要使用此选项。

小技巧

```
target:~$ dd if=/dev/zero of=/dev/mmcblk0 conv=fsync
```

还会销毁设备上的所有信息，但这个命令对磨损均衡不好，并且需要更长的时间！

7.4.7 eMMC 启动分区

eMMC 设备包含四个不同的硬件分区：User 分区、boot1 分区、boot2 分区和 rpmb 分区。

User 分区在 JEDEC 标准中称为用户数据区，是主要的存储分区。分区 boot1 和 boot2 可以用于存放 bootloader，并且更可靠。i.MX 93 使用哪个分区加载 bootloader 由 eMMC 设备的引导配置控制。分区 rpmb 是一个小分区，只能通过受信任的机制访问。

此外，User 分区可以分为四个用户定义的一般用途区域分区。对此功能的解释超出了本文件的范围。有关更多信息，请参阅 JEDEC 标准第 7.2 章分区管理。

小技巧

不要将 eMMC 分区与 DOS、MBR 或 GPT 分区表的分区混淆。

当前的 PHYTEC BSP 没有使用 eMMC 设备的额外分区功能。U-Boot 被烧写到用户分区的开始位置。U-Boot 环境被放置在 U-Boot 之后的固定位置。使用 MBR 分区表创建两个分区，一个是 FAT32 引导分区，另一个是 ext4 根文件系统分区。它们位于 U-Boot 和 U-Boot 环境之后。FAT32 引导分区包含内核和设备树。

使用 eMMC 时，可以利用专用的启动分区冗余存储 bootloader。U-Boot 环境仍然位于第一个分区之前的用户区。用户区仍然包含在初始化过程中首次装载的 bootloader。下面是一个示例，演示如何将 bootloader 刷写到两个启动分区中的一个，并通过用户空间命令切换启动设备。

7.4.8 通过用户空间命令

在主机上运行：

```
host:~$ scp imx-boot root@192.168.3.11:/tmp/
```

默认情况下，boot1 和 boot2 分区是只读的。要从用户空间写入它们，您必须在 sysfs 中禁用 force_ro。

要手动将 bootloader 写入 eMMC boot 分区，首先禁用写保护：

```
target:~$ echo 0 > /sys/block/mmcblk0boot0/force_ro
target:~$ echo 0 > /sys/block/mmcblk0boot1/force_ro
```

将 bootloader 写入 eMMC boot 分区：

```
target:~$ dd if=/tmp/imx-boot of=/dev/mmcblk0boot0
target:~$ dd if=/tmp/imx-boot of=/dev/mmcblk0boot1
```

下表是 i.MX 93 SoC 的偏移量：

SoC	User 区域偏移量	Boot 分区偏移量	eMMC 设备	bootloader 文件名
i.MX 93	32 kiB	0 千字节	/dev/mmcblk0	imx-boot

之后使用 mmc 工具从用户空间设置引导分区：

(对于'boot0')：

```
target:~$ mmc bootpart enable 1 0 /dev/mmcblk0
```

(对于'boot1')：

```
target:~$ mmc bootpart enable 2 0 /dev/mmcblk0
```

要禁用从 eMMC boot 分区启动，只需输入以下命令：

```
target:~$ mmc bootpart enable 0 0 /dev/mmcblk0
```

选择返回到用户区域的 u-boot 环境：

```
target:~$ mmc bootpart enable 7 0 /dev/mmcblk0
```

7.4.9 调整 ext4 根文件系统的大小

fdisk 可以用来扩展根文件系统。这个例子适用于任何块设备，比如 eMMC、SD 卡或硬盘。

- 获取当前设备大小：

  ```
  target:~$ fdisk -l /dev/mmcblk0
  ```

- 输出看起来像：

  ```
  Disk /dev/mmcblk0: 7264 MB, 7616856064 bytes, 14876672 sectors 116224 cylinders, 4 heads, 32 sectors/
  ↪track
  Units: sectors of 1 * 512 = 512 bytes

  Device       Boot StartCHS        EndCHS      StartLBA     EndLBA    Sectors   Size   Id Type
  ```

(续下页)

(接上页)

```
/dev/mmcblk0p1 *    128,0,1       1023,3,32    16384       140779     124396   60.7M   c Win95 FAT32␣
↪(LBA)
/dev/mmcblk0p2      1023,3,32     1023,3,32    141312      2192013    2050702  1001M  83 Linux
```

- 使用 fdisk 删除并创建一个最大容量的分区：

```
target:~$ fdisk /dev/mmcblk0

The number of cylinders for this disk is set to 116224.
There is nothing wrong with that, but this is larger than 1024,
and could in certain setups cause problems with:
1) software that runs at boot time (e.g., old versions of LILO)
2) booting and partitioning software from other OSs
   (e.g., DOS FDISK, OS/2 FDISK)

Command (m for help): p
Disk /dev/mmcblk0: 7264 MB, 7616856064 bytes, 14876672 sectors
116224 cylinders, 4 heads, 32 sectors/track
Units: sectors of 1 * 512 = 512 bytes

Device       Boot   StartCHS        EndCHS      StartLBA     EndLBA    Sectors   Size   Id Type
/dev/mmcblk0p1 *     128,0,1     1023,3,32         16384     140779     124396  60,7M    c Win95␣
↪FAT32 (LBA)
/dev/mmcblk0p2     1023,3,32     1023,3,32        141312    2192013    2050702  1001M   83 Linux

Command (m for help): d
Partition number (1-4): 2

Command (m for help): p
Disk /dev/mmcblk0: 7264 MB, 7616856064 bytes, 14876672 sectors
116224 cylinders, 4 heads, 32 sectors/track
Units: sectors of 1 * 512 = 512 bytes

Device       Boot StartCHS    EndCHS          StartLBA     EndLBA    Sectors  Size   Id Type
/dev/mmcblk0p1 *  128,0,1     1023,3,32          16384     140779     124396  60.7M   c Win95 FAT32␣
↪(LBA)

Command (m for help): n
Partition type
   p   primary partition (1-4)
   e   extended
p
Partition number (1-4): 2
First sector (32-14876671, default 32): 141456
Last sector or +size{,K,M,G,T} (141456-14876671, default 14876671):
Using default value 14876671

Command (m for help): p
Disk /dev/mmcblk0: 7264 MB, 7616856064 bytes, 14876672 sectors
116224 cylinders, 4 heads, 32 sectors/track
Units: sectors of 1 * 512 = 512 bytes

Device       Boot StartCHS      EndCHS        StartLBA     EndLBA    Sectors  Size   Id Type
/dev/mmcblk0p1 *   128,0,1      1023,3,32        16384     140779     124396  60.7M   c Win95 FAT32␣
↪(LBA)
/dev/mmcblk0p2   1023,3,32      1023,3,32       141456   14876671   14735216  7194M  83 Linux
```

可以在文件系统挂载时增加文件系统的大小。这是一个在线调整大小的操作。但您也可以从 SD 卡启动，然后在 eMMC 分区未挂载时调整其文件系统的大小。此外，必须重启板子，以便读取新的分区表。

7.5 GPIOs

phyBOARD-Segin/Nash i.MX 93 没有专门为 GPIO 分配的一组引脚，因为所有 GPIO 都被内核设备驱动程序使用或用于特定目的。处理器将其 GPIO 组织为五个 32 个 GPIO 的组（GPIO1 –GPIO4）。gpiochip0、gpiochip32、gpiochip64 和 gpiochip96 是这些内部 i.MX 93 GPIO 组 GPIO1 –GPIO4 的 sysfs 表示。

GPIO 被标识为 GPIO<X>_<Y>（例如，GPIO4_07）。<X> 标识 GPIO Bank，并从 1 到 4 计数，而 <Y> 表示 Bank 内的 GPIO。<Y> 从 0 到 31 计数（每个 Bank 有 32 个 GPIO）。

相比之下，Linux 内核使用一个单一的整数来枚举系统中所有可用的 GPIO。计算正确数字的公式是：

```
Linux GPIO number: <N> = (<X> - 1) * 32 + <Y>
```

从用户空间访问 GPIO 将使用 libgpiod。它提供了一个库和工具，用于与 Linux GPIO 字符设备进行交互。以下是一些工具的用法示例：

- 检测芯片上的 gpiochips：

```
target:~$ gpiodetect
gpiochip0 [43810080.gpio] (32 lines)
gpiochip1 [43820080.gpio] (32 lines)
gpiochip2 [43830080.gpio] (32 lines)
gpiochip3 [47400080.gpio] (32 lines)
```

备注

Order of GPIOchips in `i.MX 93 Application Processor Reference Manual` and in Linux kernel differ!

GPIOchip 地址	Linux	Reference Manual
0x43810080	gpiochip0	gpiochip2
0x43820080	gpiochip1	gpiochip3
0x43830080	gpiochip2	gpiochip4
0x47400080	gpiochip3	gpiochip1

- 显示关于 gpiochips 的详细信息，包括它们的名称、consumer、方向、活动状态和附加标志：

```
target:~$ gpioinfo gpiochip0
```

- 读取 GPIO 的值（例如，从 chip0 读取 GPIO 3）：

```
target:~$ gpioget gpiochip0 3
```

- 将 chip0 上的 GPIO 3 的值设置为 0 并退出工具：

```
target:~$ gpioset --mode=exit gpiochip0 3=0
```

- gpioset 的帮助文本显示了可能的选项：

```
target:~$ gpioset --help
Usage: gpioset [OPTIONS] <chip name/number> <offset1>=<value1> <offset2>=<value2> ...
Set GPIO line values of a GPIO chip
```

（续下页）

(接上页)

```
Options:
  -h, --help:           display this message and exit
  -v, --version:        display the version and exit
  -l, --active-low:     set the line active state to low
  -m, --mode=[exit|wait|time|signal] (defaults to 'exit'):
                tell the program what to do after setting values
  -s, --sec=SEC:        specify the number of seconds to wait (only valid for --mode=time)
  -u, --usec=USEC:      specify the number of microseconds to wait (only valid for --mode=time)
  -b, --background:     after setting values: detach from the controlling terminal

Modes:
  exit:         set values and exit immediately
  wait:         set values and wait for user to press ENTER
  time:         set values and sleep for a specified amount of time
  signal:       set values and wait for SIGINT or SIGTERM

Note: the state of a GPIO line controlled over the character device reverts to default
when the last process referencing the file descriptor representing the device file exits.
This means that it's wrong to run gpioset, have it exit and expect the line to continue
being driven high or low. It may happen if given pin is floating but it must be interpreted
as undefined behavior.
```

警告

某些 GPIO 用于特殊功能。在使用某个 GPIO 之前，请参考您板子的原理图或硬件手册，以确保该 IO 未被其他功能占用。

7.5.1 通过 sysfs 访问 GPIO

警告

通过 sysfs 访问 GPIO 已被弃用，我们建议使用 libgpiod。

默认情况下不再启用通过 sysfs 访问 GPIO 的支持。只有手动在内核配置中启用 CONFIG_GPIO_SYSFS 后才能实现。要使 CONFIG_GPIO_SYSFS 在 menuconfig 中可见，必须先启用 CONFIG_EXPERT 选项。

您还可以将此选项添加到 Linux 内核源代码中 arch/arm64/configs 下的 imx9_phytec_distro.config 配置片段中，例如：

```
..
CONFIG_GPIO_SYSFS=y
..
```

7.6 ADC

PHYTEC i.MX 93 包含通用的 ADC，可用于与模拟传感器连接。

通过 sysfs 可以读取 ADC 值：

```
target:~$ cat /sys/bus/iio/devices/iio:deviceX/in_voltageY_raw
```

在 phyBOARD-Nash 上，ADC 线路可以通过 X16 扩展连接器访问：

ADC 输入	X16 引脚
ADC_IN0	47
ADC_IN2	49

7.7 LED 灯

如果有任何 LED 连接到 GPIO，您可以通过一个特殊的 LED 驱动程序接口访问它们，而不是使用通用的 GPIO 接口（请参见 GPIO 部分）。您将通过 `/sys/class/leds/` 而不是 `/sys/class/gpio/` 来访问它们。LED 的最大亮度可以从 `max_brightness` 文件中读取。亮度文件将设置 LED 的亮度（取值范围从 0 到 max_brightness）。大多数 LED 没有硬件亮度支持，只会在所有非零亮度设置下点亮。

下面是一个简单的例子。

要获取所有可用的 LED，请输入：

```
target:~$ ls /sys/class/leds
green:heartbeat@  mmc0::@  mmc1::@  yellow:@
```

这里的 LED 灯 `green:heartbeat` 位于 phyCORE-i.MX93 上。如果您使用的是 phyBOARD-Segin，PEB-EVAL-01 上还有一个 黄色 LED 灯。

- 要打开 LED 灯：

  ```
  target:~$ echo 1 > /sys/class/leds/green\:heartbeat/brightness
  ```

- 要关闭：

  ```
  target:~$ echo 0 > /sys/class/leds/green\:heartbeat/brightness
  ```

GPIO 配置的设备树配置可以在这里找到：https://git.phytec.de/linux-imx/tree/arch/arm64/boot/dts/freescale/imx93-phyboard-segin-peb-eval-01.dtso?h=v6.1.55_2.2.0-phy3#n33

7.8 I²C 总线

该 i.MX 93 包含多个多主机 fast-mode I²C 模块。PHYTEC 板提供了许多不同的 I²C 设备，这些设备连接到 i.MX 93 的 I²C 模块。本节描述了我们 phyBOARD-Segin/Nash i.MX 93 中集成的一些 I²C 设备的基本设备使用及其设备树（DT）表示。

i2c 的设备树节点包含一些设置，例如时钟频率，用于设置总线频率，以及引脚控制设置，包括 scl-gpios 和 sda-gpios，这些是用于总线恢复的备用引脚配置。

I²C1 总线 DT 配置（例如 imx93-phycore-som.dtsi）：https://git.phytec.de/linux-imx/tree/arch/arm64/boot/dts/freescale/imx93-phycore-som.dtsi?h=v6.1.55_2.2.0-phy3#n88

imx93-phyboard-segin.dts 的 I²C2 总线 DT 配置：https://git.phytec.de/linux-imx/tree/arch/arm64/boot/dts/freescale/imx93-phyboard-segin.dts?h=v6.1.55_2.2.0-phy3#n155 或者 imx93-phyboard-nash.dts: https://git.phytec.de/linux-imx/tree/arch/arm64/boot/dts/freescale/imx93-phyboard-nash.dts?h=v6.1.55_2.2.0-phy3#n113

7.9 EEPROM

在 phyCORE-i.MX93 SoM 和 phyBOARD-Segin/Nash i.MX 93 上有两个不同的 I2C EEPROM 闪存。目前只有 phyCORE-i.MX93 上的闪存被启用，并且它用于板检测。

7.9.1 phyCORE-i.MX93 上的 I2C EEPROM

phyCORE-i.MX93 SoM 上的 I2C EEPROM 的空间被划分两部分。

- 正常区域（大小：4096 字节，总线：I2C-2，地址：0x50）
- ID 页面（大小：32 字节，总线：I2C-2，地址：0x58）

可以从设备进行读写操作：

```
target:~$ hexdump -C /sys/class/i2c-dev/i2c-2/device/2-0058/eeprom
```

要读取并以十六进制打印 EEPROM 的前 1024 字节，请执行：

```
target:~$ dd if=/sys/class/i2c-dev/i2c-2/device/2-0050/eeprom bs=1 count=1024  | od -x
```

要将整个 EEPROM（ID 页）填充为零，我们首先需要禁用 EEPROM 写保护，请使用：

```
target:~$ gpioset 2 21=0
```

然后可以写入 EEPROM：

```
target:~$ dd if=/dev/zero of=/sys/class/i2c-dev/i2c-2/device/2-0058/eeprom bs=32 count=1
```

警告

正常 EEPROM 区域的前 256 个字节（总线：I2C-2 地址：0x50）是保留的，不应被覆盖！（见下文）

7.9.2 EEPROM SoM 检测

PHYTEC 在 EEPROM 正常区域的前 256 字节中存储有关 SoM 的信息。这包括 PCB 版本和贴装选项。

在启动的早期阶段读取 EEPROM 数据。它用于选择正确的 DDR RAM 配置。这使得可以使用相同的 bootloader 镜像来支持不同的 RAM 大小，并自动选择正确的 DTS overlay。

如果正常区域的前 265 个字节被删除，启动加载程序将回退到 phyCORE-i.MX93 开发板内存配置，即 1GiB RAM。

警告

正常 EEPROM 区域的前 265 个字节（总线：I2C-2 地址：0x50）不应被擦除或损坏！这可能会影响 bootloader 的行为。板子可能无法正确启动。

7.9.3 恢复 EEPROM 数据

硬件自检数据已预先写入 EEPROM 数据空间。如果您不小心删除或覆盖了数据，请联系我们的支持团队！

DT 表示，例如在 phyCORE-i.MX 93 文件中可以在我们的 PHYTEC git 中找到：https://git.phytec.de/linux-imx/tree/arch/arm64/boot/dts/freescale/imx93-phycore-som.dtsi?h=v6.1.55_2.2.0-phy3#n172

7.10 RTC

RTC 可以通过 `/dev/rtc*` 访问。由于 PHYTEC 板通常有多个 RTC，因此可能会有多个 RTC 设备文件。

- 要找到 RTC 设备的名称，可以通过以下方式读取其 sysfs 条目：

```
target:~$ cat /sys/class/rtc/rtc*/name
```

- 例如，你将得到：

```
rtc-rv3028 0-0052
snvs_rtc 30370000.snvs:snvs-rtc-lp
```

小技巧

这将列出所有实时时钟（RTC），包括非 I²C 接口的 RTC。如果存在设备树/aliases 条目，Linux 会根据这些条目分配 RTC 设备 ID。

日期和时间可以通过 `hwclock` 工具和 date 命令进行操作。要显示目标上设置的当前日期和时间：

```
target:~$ date
Thu Jan  1 00:01:26 UTC 1970
```

使用日期命令更改日期和时间。日期命令以以下语法设置时间："YYYY-MM-DD hh:mm:ss (+|-)hh:mm"：

```
target:~$ date -s "2022-03-02 11:15:00 +0100"
Wed Mar  2 10:15:00 UTC 2022
```

备注

您的时区（在此示例中为 +0100）可能会有所不同。

使用 date 命令并不会改变实时时钟（RTC）的时间和日期，因此如果我们重启目标设备，这些更改将会被丢弃。要写入 RTC，我们需要使用 `hwclock` 命令。使用 `hwclock` 工具将当前的日期和时间（通过 date 命令设置）写入 RTC，然后重启目标设备以检查更改是否已应用到 RTC 上：

```
target:~$ hwclock -w
target:~$ reboot
    .
    .
    .
target:~$ date
Wed Mar  2 10:34:06 UTC 2022
```

要从实时时钟（RTC）设置系统时间和日期，请使用：

```
target:~$ date
Thu Jan  1 01:00:02 UTC 1970
target:~$ hwclock -s
target:~$ date
Wed Mar  2 10:45:01 UTC 2022
```

7.10.1 RTC 唤醒报警器

可以从实时时钟（RTC）发出中断以唤醒系统。该格式使用 Unix 纪元时间，即自 1970 年 1 月 1 日 UTC 午夜以来的秒数。要在从挂起到 RAM 状态后的 4 分钟唤醒系统，请输入：

```
target:~$ echo "+240" > /sys/class/rtc/rtc0/wakealarm
target:~$ echo mem > /sys/power/state
```

备注

内部唤醒警报时间将被向上舍入到下一个分钟，因为警报功能不支持秒。

7.10.2 RTC 参数

实时时钟（RTC）具有一些功能，可以通过 `hwclock` 工具进行读取和设置。

- 我们可以通过以下方式检查 RTC 支持的功能：

  ```
  target:~$ hwclock --param-get features
  The RTC parameter 0x0 is set to 0x71.
  ```

  这个值的含义在内核中进行了编码，每个位被设置为 1 的位翻译为：

  ```
  #define RTC_FEATURE_ALARM               0
  #define RTC_FEATURE_ALARM_RES_MINUTE    1
  #define RTC_FEATURE_NEED_WEEK_DAY       2
  #define RTC_FEATURE_ALARM_RES_2S        3
  #define RTC_FEATURE_UPDATE_INTERRUPT    4
  #define RTC_FEATURE_CORRECTION          5
  #define RTC_FEATURE_BACKUP_SWITCH_MODE  6
  #define RTC_FEATURE_ALARM_WAKEUP_ONLY   7
  #define RTC_FEATURE_CNT                 8
  ```

- 我们可以通过以下方式检查 RTC BSM（备份切换模式）：

  ```
  target:~$ hwclock --param-get bsm
  The RTC parameter 0x2 is set to 0x1.
  ```

- 我们可以通过以下方式设置 RTC BSM：

  ```
  target:~$ hwclock --param-set bsm=0x2
  The RTC parameter 0x2 will be set to 0x2.
  ```

  BSM 值的含义翻译为以下这些值：

  ```
  #define RTC_BSM_DISABLED   0
  #define RTC_BSM_DIRECT     1
  #define RTC_BSM_LEVEL      2
  #define RTC_BSM_STANDBY    3
  ```

  小技巧

  您应该将 BSM 模式设置为 DSM 或 LSM，以便在初始电源不可用时，RTC 可以切换到备用电源。请查看 **RV-3028** RTC 数据表，以了解 LSM（电平切换模式）和 DSM（直接切换模式）的实际含义。

I²C RTC 的设备树表示：https://git.phytec.de/linux-imx/tree/arch/arm64/boot/dts/freescale/imx93-phyboard-segin.dts?h=v6.1.55_2.2.0-phy3#n173 或 https://git.phytec.de/linux-imx/tree/arch/arm64/boot/dts/freescale/imx93-phyboard-nash.dts?h=v6.1.55_2.2.0-phy3#n122

7.11 USB 主控制器

i.MX 93 SoC 的 USB 控制器为众多消费类便携设备提供了一种低成本的连接解决方案，通过提供一种数据传输机制，使 USB 设备之间的速度高达 480 Mbps（高速'HS'）。USB 子系统具有两个独立的 USB 控制器核心。两个控制器都能够充当 USB Device 或 USB Host，但在 phyBOARD-Segin/Nash i.MX 93 上，其中一个被用作仅 Host 端口（USB-A 连接器）。

统一的 BSP 支持大容量存储设备和键盘。其他与 USB 相关的设备驱动程序必须根据需要在内核配置中启用。由于 udev，所有连接的存储设备都会获得唯一的 ID，并可以在 `/dev/disk/by-id` 中找到。这些 ID 可以在 `/etc/fstab` 中用于以不同的方式挂载不同的 USB 存储设备。

OTG 端口提供了一个额外的引脚用于过流保护，但在 phyBOARD-Segin/Nash i.MX 93 上并未使用。由于未使用，该引脚在设备树中也被禁用。如果需要使用该引脚，请在设备树中激活过流保护，并根据设备规格设置正确的极性（高电平/低电平）。有关正确的设置，请参考 Linux 内核文档中的 Linux/Documentation/devicetree/bindings/usb/ci-hdrc-usb2.txt。

USB Host 的 DT 表示：https://git.phytec.de/linux-imx/tree/arch/arm64/boot/dts/freescale/imx93-phyboard-segin.dts?h=v6.1.55_2.2.0-phy3#n190 或 https://git.phytec.de/linux-imx/tree/arch/arm64/boot/dts/freescale/imx93-phyboard-nash.dts?h=v6.1.55_2.2.0-phy3#n180

7.12 RS232/RS485

phyBOARD-Nash i.MX 93 SoC 提供一个 RS232/RS485 串口。

警告

由于 phyBOARD-Nash PCB 版本 1616.0 上的硬件缺陷，RS232 的硬件流控制和 RS485 无法正常工作

7.12.1 RS232

- 以人类可读的格式显示终端的当前设置：

```
target:~$ stty -a
```

- UART 接口的配置可以通过 stty 命令完成。例如：

```
target:~$ stty -F /dev/|serial-uart| 115200 crtscts raw -echo
```

- 通过简单的 echo 和 cat，可以测试基本的通信。示例：

```
target:~$ echo 123 > /dev/|serial-uart|
```

```
host:~$ cat /dev/ttyUSB2
```

主机应该打印出"123"。

7.12.2 RS485

为了方便测试，请查看 linux-serial-test。这个工具会通过调用 RS485 的 IOCTL，发送恒定的数据流。

```
target:~$ linux-serial-test -p /dev/|serial-uart| -b 115200 --rs485 0
```

有关 linux-serial-test 工具及其参数的更多信息，请访问此链接：linux-serial-test

在 Linux 内核文档中描述了在如何在 C 代码中调用 IOCTL：https://www.kernel.org/doc/Documentation/serial/serial-rs485.txt

RS232 和 RS485 的设备树表示：https://git.phytec.de/linux-imx/tree/arch/arm64/boot/dts/freescale/imx93-phyboard-nash.dts?h=v6.1.55_2.2.0-phy3#n173

7.13 CAN FD

phyBOARD-Segin/Nash i.MX 93 具有一个支持 CAN FD 的 flexCAN 接口。它们由 Linux 标准 CAN 框架支持，该框架建立在 Linux 网络层之上。使用该框架，CAN 接口表现得像普通的 Linux 网络设备，并具有一些特定于 CAN 的附加功能。更多信息可以在 Linux 内核文档中找到：https://www.kernel.org/doc/html/latest/networking/can.html

- 使用：

  ```
  target:~$ ip link
  ```

  查看接口的状态。CAN 接口应该显示为 can0。

- 要获取有关 can0 的信息，例如比特率和错误计数器，请输入：

  ```
  target:~$ ip -d -s link show can0
  4: can0: <NOARP,UP,LOWER_UP,ECHO> mtu 16 qdisc pfifo_fast state UP mode DEFAULT group default qlen 10
     link/can  promiscuity 0  allmulti 0 minmtu 0 maxmtu 0
     can state ERROR-ACTIVE (berr-counter tx 0 rx 0) restart-ms 0
           bitrate 500000 sample-point 0.875
           tq 25 prop-seg 37 phase-seg1 32 phase-seg2 10 sjw 1 brp 1
           flexcan: tseg1 2..96 tseg2 2..32 sjw 1..16 brp 1..1024 brp_inc 1
           flexcan: dtseg1 2..39 dtseg2 2..8 dsjw 1..4 dbrp 1..1024 dbrp_inc 1
           clock 40000000
           re-started bus-errors arbit-lost error-warn error-pass bus-off
           0          0          0          0          0          0         numtxqueues 1 numrxqueues 1␣
  ↪gso_max_size 65536 gso_max_segs 65535 tso_max_size 65536 tso_max_segs 65535 gro_max_size 65536␣
  ↪parentbus platform parentdev 443a0000.can
     RX:  bytes packets errors dropped  missed   mcast
              0       0      0       0       0       0
     TX:  bytes packets errors dropped carrier collsns
              0       0      0       0       0       0
  ```

输出包含一组标准参数，这些参数也适用于以太网接口，因此并非所有参数对于 CAN 都是相关的（例如 MAC 地址）。以下输出参数包含有用的信息：

can0	接口名称
NOARP	CAN 无法使用 ARP 协议
MTU	最大传输单元
RX packets	接收的数据包数量
TX packets	发送的数据包数量
RX bytes	接收字节数
TX bytes	发送字节数
errors...	总线错误统计信息

CAN 配置是在 systemd 配置文件 `/lib/systemd/network/can0.network` 中完成的。为了持久化更改（例如，默认比特率），请在 BSP 中更改根文件系统下的 `./meta-ampliphy/recipes-core/systemd/systemd-conf/can0.network` 中的配置，并重新构建根文件系统。

```
[Match]
Name=can0

[Can]
BitRate=500000
```

比特率也可以手动更改，例如，以利用灵活的比特率：

```
target:~$ ip link set can0 down
target:~$ ip link set can0 txqueuelen 10 up type can bitrate 500000 sample-point 0.75 dbitrate 4000000␣
↪dsample-point 0.8 fd on
```

您可以使用 cansend 发送消息，或使用 candump 接收消息：

```
target:~$ cansend can0 123#45.67
target:~$ candump can0
```

要生成用于测试目的的随机 CAN 流量，请使用 cangen：

```
target:~$ cangen
```

`cansend --help` 和 `candump --help` 提供了关于选项和用法的帮助信息。

imx93-phyboard-segin.dts 的设备树 CAN 配置：https://git.phytec.de/linux-imx/tree/arch/arm64/boot/dts/freescale/imx93-phyboard-segin.dts?h=v6.1.55_2.2.0-phy3#n147 或 https://git.phytec.de/linux-imx/tree/arch/arm64/boot/dts/freescale/imx93-phyboard-nash.dts?h=v6.1.55_2.2.0-phy3#n105

7.14 phyBOARD-Segin 上的音频

在 phyBOARD-Segin i.MX 93 上使用的是 TI TLV320AIC3007 音频编解码器。它使用 I2S 进行数据传输，使用 I2C 进行编解码器控制。可用的音频信号包括：

- 立体声 LINE IN，
- 立体声 LINE OUT，
- Class D 1W 的扬声器输出

要检查您的声卡驱动程序是否正确加载以及设备名称，请输入以下命令以查看播放设备：

```
target:~$ aplay -L
```

或输入录音设备：

```
target:~$ arecord -L
```

7.14.1 Alsamixer

要检查声卡的功能，请调用：

```
target:~$ alsamixer
```

您应该会看到很多选项，因为音频 IC 具有许多可以测试的功能。通过 SSH 打开 alsamixer 可能更好，而不是通过串行控制台，因为控制台的图形效果更好。所有混音点都有单声道或立体声增益控制。"MM" 表示该功能被静音（左右输出均为静音），可以通过按' **m** ' 切换。您还可以通过按' < ' for left and ' > ' 切换右声道输出。使用 **tab** 键，您可以在播放和录音控制之间切换。

7.14.2 恢复默认音量

一些默认设置存储在 `/var/lib/alsa/asound.state` 中。您可以使用以下命令保存当前的 alsa 设置：

```
target:~$ alsactl --file </path/to/filename> store
```

您可以通过以下方式恢复已保存的 alsa 设置：

```
target:~$ alsactl --file </path/to/filename> restore
```

7.14.3 播放

运行 speaker-test 以检查播放功能：

```
target:~$ speaker-test -c 2 -t wav
```

要播放简单的音频流，您可以使用 aplay。例如，要播放 ALSA 测试音效：

```
target:~$ aplay /usr/share/sounds/alsa/*
```

要播放其他格式，例如 mp3，您可以使用 Gstreamer：

```
target:~$ gst-launch-1.0 playbin uri=file:/path/to/file.mp3
```

如果扬声器音量太低，可以增加音量（值范围 0-3）：

```
target:~$ amixer -c 0 sset Class-D 3
```

提示

扬声器输出仅为单声道，因此当播放立体声轨道时，扬声器只会播放左声道。

7.14.4 捕获

`arecord` 是一个命令行工具，用于捕获音频流，默认输入源为线路输入（Line In）。

```
target:~$ arecord -t wav -c 2 -r 44100 -f S16_LE test.wav
```

提示

由于播放和录音共享硬件接口，因此无法在同时进行播放和录音操作时使用不同的采样率和格式。

音频设备树配置：https://git.phytec.de/linux-imx/tree/arch/arm64/boot/dts/freescale/imx93-phyboard-segin.dts?h=v6.1.55_2.2.0-phy3#n62

7.15 phyBOARD-Nash 上的音频

警告

由于硬件缺陷，phyBOARD-Nash i.MX 93 PCB 版本：1616.0 上的音频功能失效

要在 phyBOARD-Nash 上使用音频，需要一个额外的扩展板用于音频/视频连接器。PEB-AV-10（1531.1 修订版）可以单独购买，而不是随套件一起提供。PEB-AV-10 配备了 TI TLV320AIC3007 音频编码解码器。音频支持通过 I2S 接口实现，并通过 I2C 控制。

有一个符合 OMTP 标准的 3.5mm 耳机插孔和一个 8 针接口，用于连接带有 AV 连接器的音频设备。这个 8 针接口包含单声道扬声器、耳机和线路输入信号。

要检查您的声卡驱动程序是否正确加载以及设备名称，请输入以下命令以查看播放设备：

```
target:~$ aplay -L
```

或输入录音设备：

```
target:~$ arecord -L
```

7.15.1 Alsamixer

要检查声卡的功能，请调用：

```
target:~$ alsamixer
```

您应该会看到很多选项，因为音频 IC 具有许多可以测试的功能。通过 SSH 打开 alsamixer 可能更好，而不是通过串行控制台，因为控制台的图形效果更好。所有混音点都有单声道或立体声增益控制。"MM" 表示该功能被静音（左右输出均为静音），可以通过按' **m** ' 切换。您还可以通过按' < ' for left and ' > ' 切换右声道输出。使用 **tab** 键，您可以在播放和录音控制之间切换。

7.15.2 恢复默认音量

一些默认设置存储在 `/var/lib/alsa/asound.state` 中。您可以使用以下命令保存当前的 alsa 设置：

```
target:~$ alsactl --file </path/to/filename> store
```

您可以通过以下方式恢复已保存的 alsa 设置：

```
target:~$ alsactl --file </path/to/filename> restore
```

7.15.3 ALSA 配置

我们的 BSP 附带一个 ALSA 配置文件 `/etc/asound.conf` 。

ALSA 配置文件可以根据需要进行编辑或删除，因为它并不是 ALSA 正常工作所必需的。

```
target:~$ vi /etc/asound.conf
```

要将 PEB-AV-10 设置为输出，请将 *playback.pcm* 从"dummy" 设置为"pebav10":

```
[...]

pcm.asymed {
        type asym
        playback.pcm "pebav10"
        capture.pcm "dsnoop"
}

[...]
```

如果声音不可听，请将播放设备更改为软件音量控制播放设备，将 *playback.pcm* 设置为相应的软音量播放设备，例如 "softvol_pebav10"。使用 alsamixer 控制来调整音量级别。

```
[...]

pcm.asymed {
        type asym
        playback.pcm "softvol_pebav10"
        capture.pcm "dsnoop"
}

[...]
```

7.15.4 PulseAudio 配置

对于使用 *Pulseaudio* 的应用程序，请检查可用的音频输出设备:

```
target:~$ pactl list short sinks
```

要选择输出设备，请输入:

```
target:~$ pactl set-default-sink <sink_number>
```

7.15.5 播放

运行 speaker-test 以检查播放功能:

```
target:~$ speaker-test -c 2 -t wav
```

要播放简单的音频流，您可以使用 aplay。例如，要播放 ALSA 测试音效:

```
target:~$ aplay /usr/share/sounds/alsa/*
```

要播放其他格式，例如 mp3，您可以使用 Gstreamer:

```
target:~$ gst-launch-1.0 playbin uri=file:/path/to/file.mp3
```

7.15.6 捕获

`arecord` 是一个命令行工具，用于捕获音频流，默认输入源为线路输入。要选择不同的音频源，可以使用 `alsamixer`。例如，打开 *右侧 PGA 混音器 Mic3R* 和 *左侧 PGA 混音器 Mic3R*，以便通过 3.5mm 插孔捕获来自 TLV320 编解码器的麦克风输入音频。

```
target:~$ amixer -c "sndpebav10" sset 'Left PGA Mixer Mic3R' on
target:~$ amixer -c "sndpebav10" sset 'Right PGA Mixer Mic3R' on
```

```
target:~$ arecord -t wav -c 2 -r 44100 -f S16_LE test.wav
```

提示

由于播放和录音共享硬件接口，因此无法在同时进行播放和录音操作时使用不同的采样率和格式。

设备树音频配置：https://git.phytec.de/linux-imx/tree/arch/arm64/boot/dts/freescale/imx93-phyboard-nash-peb-av-010.dtso?h=v6.1.55_2.2.0-phy3#n57

7.16 视频

7.16.1 使用 Gstreamer 的视频

默认情况下，在 BSP 中安装了一个示例视频，路径为 */usr/share/qtphy/videos/* 。可以使用以下命令之一开始视频播放：

```
target:~$ gst-launch-1.0 playbin uri=file:///usr/share/qtphy/videos/caminandes_3_llamigos_720p_vp9.webm
```

- 或者：

```
target:~$ gst-launch-1.0 -v filesrc location=/usr/share/qtphy/videos/caminandes_3_llamigos_720p_vp9.webm !
↪decodebin name=decoder decoder. ! videoconvert ! waylandsink
```

- 或者：

```
target:~$ gst-play-1.0 /usr/share/qtphy/videos/caminandes_3_llamigos_720p_vp9.webm --videosink waylandsink
```

7.17 显示

phyBOARD-Segin i.MX 93 支持 PEB-AV-02，配备 7 英寸的 `edt,etm0700g0edh6` 并口显示屏和电容触摸屏。该显示屏的设备树 overlay 在 `BOOT/bootenv.txt` 中默认启用！

phyBOARD-Nash i.MX 93 需要额外的适配板来支持 10 英寸的 `edt,etml1010g3dra` LVDS 显示器和电容触摸。PEB-AV-10（1531.1 修订版）可以单独购买。该显示器的 overlay 在 `BOOT/bootenv.txt` 中默认启用！

7.17.1 Qt Demo

使用 `phytec-qt6demo-image` 时，Weston 会在启动时启动。我们的 Qt6 DEMO 应用程序名为 “qtphy”，可以通过以下方式停止：

```
target:~$ systemctl stop qtphy
```

- 要重新开始 Demo，请运行：

```
target:~$ systemctl start qtphy
```

- 要禁用 Demo 的自动启动，请运行：

```
target:~$ systemctl disable qtphy
```

- 要启用 Demo 的自动启动，请运行：

```
target:~$ systemctl enable qtphy
```

- Weston 可以通过以下方式停止：

```
target:~$ systemctl stop weston
```

备注

在关闭 Weston 之前，必须先关闭 Qt Demo。

7.17.2 背光控制

如果显示器连接到 PHYTEC 板，可以通过 Linux 内核的 sysfs 接口控制其背光。系统中所有可用的背光设备可以在文件夹/sys/class/backlight 中找到。读取相应的文件并向其写入数据可以控制背光。

备注

Some boards with multiple display connectors might have multiple backlight controls in /sys/class/backlight. For example: backlight0 and backlight1

- 例如，要获取最大亮度级别（max_brightness），请执行：

```
target:~$ cat /sys/class/backlight/backlight/max_brightness
```

  有效的亮度值范围是 0 到 <max_brightness>。

- 要获取当前亮度级别，请输入：

```
target:~$ cat /sys/class/backlight/backlight/brightness
```

- 写入文件 brightness 以更改亮度：

```
target:~$ echo 0 > /sys/class/backlight/backlight/brightness
```

  例如，关闭背光。

  有关所有文件的文档，请参见 https://www.kernel.org/doc/Documentation/ABI/stable/sysfs-class-backlight。

PEB-AV-02 的设备树可以在这里找到：https://git.phytec.de/linux-imx/tree/arch/arm64/boot/dts/freescale/imx93-phyboard-segin-peb-av-02.dtso?h=v6.1.55_2.2.0-phy3

PEB-AV-10 的设备树可以在这里找到：https://git.phytec.de/linux-imx/tree/arch/arm64/boot/dts/freescale/imx93-phyboard-nash-peb-av-010.dtso?h=v6.1.55_2.2.0-phy3

7.18 电源管理

7.18.1 CPU 核心频率调节

i.MX 93 SoC 中的 CPU 能够调整时钟频率和电压。这用于在不需要 CPU 的全部性能时节省电力。与 i.MX8 M 系列不同，i.MX 93 不支持 动态电压和频率调整（DVFS），但支持简化的 **电压和频率调整（VFS）**。该板可以进入以下模式：

- nominal（ND），
- overdrive （OD），
- Low Drive（LD）和
- Low Drive（LD）与软件快速频率变化（SWFFC）。

模式	CPU 频率	DDR 数据速率	VDD_SOC
OverDrive (OD)	1.7 GHz	3733 MT/s	900mV
NominalDrive (ND)	1.4 GHz	1866 MT/s	850mV
LowDrive (LD)	900 MHz	1866 MT/s	800mV
带有 SWFFC 的 LowDrive（LD）	900 MHz	625 MT/s	800mV

i.MX 93 BSP 支持 VFS 功能。Linux 内核提供了一个 LPM 驱动程序，可以设置 VDD_SOC、CPU 频率和 DDR 速度。

- 要将设备置于 **OverDrive（OD）**模式，请输入：

```
target:~$ echo 0 > /sys/devices/platform/imx93-lpm/mode
```

- 要将设备置于 **NominalDrive（ND）**模式，请输入：

```
target:~$ echo 1 > /sys/devices/platform/imx93-lpm/mode
```

- 要将设备置于 **LowDrive（LD）**模式，请输入：

```
target:~$ echo 2 > /sys/devices/platform/imx93-lpm/mode
```

- 将设备置于 **LowDrive（LD）**模式，使用最低 DDR 速度，SWFFC 类型：

```
target:~$ echo 3 > /sys/devices/platform/imx93-lpm/mode
```

- 要检查当前的 CPU 频率，请输入：

```
target:~$ mhz
```

- 要检查当前模式和 DDR 频率，请输入：

```
target:~$ cat /sys/devices/platform/imx93-lpm/mode
```

- 要检查当前的 VDD_SOC 类型：

```
target:~$ cat /sys/kernel/debug/regulator/regulator_summary
```

有关 LPM 驱动程序和模式的更详细信息，请参考 NXP 的文档：https://docs.nxp.com/bundle/AN13917/page/topics/low_power_mode_use_cases.html

7.18.2 CPU 核心管理

i.MX 93 SoC 在芯片上拥有多个处理器核心。例如，i.MX 93 具有 2 个 ARM 核，这些核可以在运行时单独开启和关闭。

- 要查看系统中所有可用的核心，请执行：

```
target:~$ ls /sys/devices/system/cpu  -l
```

- 这将显示，例如：

```
cpu0/
cpu1/
cpufreq/
[...]
```

  这里系统有两个处理器核心。默认情况下，系统中所有可用的核心都被启用，以获得最佳性能。

- 要关闭某个核，请执行：

```
target:~$ echo 0 > /sys/devices/system/cpu/cpu1/online
```

  作为确认，您将看到：

```
[  110.505012] psci: CPU1 killed (polled 0 ms)
```

  现在核心已关闭电源，并且该核心上不再调度任何进程。

- 您可以使用 top 命令查看核心和进程的图形概览：

```
target:~$ htop
```

- 要重新启动核心，请执行：

```
target:~$ echo 1 > /sys/devices/system/cpu/cpu1/online
```

7.18.3 挂起到 RAM

phyCORE-i.MX93 支持基本的挂起和恢复。可以使用不同的唤醒源。挂起/恢复可以通过以下方式实现：

```
target:~$ echo mem > /sys/power/state
#resume with pressing on/off button
```

要通过串行控制台唤醒，请运行

```
target:~$ echo enabled > /sys/class/tty/ttyLP0/power/wakeup
target:~$ echo mem > /sys/power/state
```

设备可以通过 PEB-EVAL-01 的 S2 按钮进入休眠状态并唤醒

要通过 RTC 闹钟唤醒，请检查：*RTC Wakealarm*

7.19 热管理

7.19.1 U-Boot

之前 U-Boot 中的温度控制不够理想。现在，U-Boot 增加了温度关机功能，以防止在启动过程中板子过热。关机发生的温度与内核中的温度一致。

当前温度的各个温度范围在启动日志中显示：

```
CPU:   Industrial temperature grade (-40C to 105C) at 33C
```

7.19.2 内核

Linux 内核集成了热管理功能，能够监测系统芯片（SoC）温度，降低 CPU 频率，控制风扇，通知其他驱动程序减少功耗，并在最坏的情况下优雅地关闭系统（https://www.kernel.org/doc/Documentation/thermal/sysfs-api.txt）。

本节描述了如何在 i.MX 93 SoC 平台上使用热管理内核 API。i.MX 9 具有用于 SoC 的内部温度传感器。

- 当前温度可以以毫摄氏度为单位读取：

  ```
  target:~$ cat /sys/class/thermal/thermal_zone0/temp
  ```

- 例如，你将得到：

  ```
  49000
  ```

imx_thermal 内核驱动注册了两个温度阈值。这些阈值根据 CPU 变种的不同而有所区别。区分包括商业级、工业级和扩展工业级。

	商业	工业	扩展工业级
被动（警告）	85°C	95°C	115°C
关键（关机）	90°C	100°C	120°C

（请查看内核 sysfs 文件夹 `/sys/class/thermal/thermal_zone0/`）

内核热管理使用这些触发点来触发事件并改变冷却行为。内核中可用的热政策（也称为 thermal governor）包括：Step Wise 和 Power Allocator。BSP 中使用的默认政策是 step_wise。

小技巧

如果 sysfs 文件中的 SoC 温度值达到 *trip_point_1* ，主板将立即关闭以避免任何热损伤。如果这不符合您的期望，可以实现一个外部监控电路，当温度降到选定的触发点以下时，通过 X_ONOFF 信号重新启动模块

备注

由于我们安装了不同温度等级的 CPU，因此热触发点的实际值可能会有所不同。

7.20 看门狗

PHYTEC i.MX 93 模块包含一个硬件看门狗，当系统挂起时能够重置板子。看门狗在 U-Boot 中默认启动，超时时间为 60 秒。因此，即使在早期内核启动过程中，看门狗也已经开始运行。Linux 内核驱动程序控制看门狗，并确保它有被踢到。本节将解释如何使用 systemd 在 Linux 中配置看门狗，以检查系统挂起和重启期间的情况。

7.20.1 systemd 中的看门狗支持

Systemd 从版本 183 开始支持硬件看门狗。

- 要启用看门狗支持，需要通过启用选项来配置 `/etc/systemd/` 中的文件 system.conf 文件：

```
RuntimeWatchdogSec=60s
ShutdownWatchdogSec=10min
```

RuntimeWatchdogSec 定义了看门狗的超时时间，而 *ShutdownWatchdogSec* 定义了系统重启时的超时时间。有关 systemd 下硬件看门狗的更多详细信息，请访问 http://0pointer.de/blog/projects/watchdog.html。更改将在重启后生效，或者运行：

```
target:~$  systemctl daemon-reload
```

7.21 bbnsm 电源键

连接到开/关按钮的 X_ONOFF 引脚可以长按（5 秒）以触发关机，而无需软件干预，或者用于唤醒系统以退出挂起状态。使用 *bbnsm_pwrkey* 驱动程序时，当按钮被按下时，KEY_POWER 事件也会报告给用户空间。默认情况下，systemd 被配置为忽略此类事件。未配置无软件干预的关机功能。可以在 `/etc/systemd/logind.conf` 中配置在按下开/关按钮时通过 systemd 触发关机，并使用以下设置：

```
HandlePowerKey=poweroff
```

7.22 PXP

i.MX 93 SoC 包含一个 PiXel Pipeline (PXP)。PXP 将以下内容组合成一个单独的处理引擎：

- 缩放
- 颜色空间转换 (CSC)
- 辅助色彩空间转换 (CSC2)
- 旋转

因此，减少了显示管道所需的内存占用。有关如何在 Gstreamer 和 Wayland 中使用 PXP，请查看 NXP 的《How to Use PXP in GStreamer and Wayland》（AN13829）应用笔记。

7.23 片上一次性可编程控制器 (OCOTP_CTRL) - eFuse

该 i.MX 93 提供一次性可编程寄存器（fuse），用于存储信息，例如 MAC 地址、启动配置和其他永久设置（在 i.MX 93 参考手册中称为“片上一次性可编程控制器（OCOTP_CTRL）”）。以下列表是 i.MX 93 参考手册的摘要，包括 OCOTP_CTRL 中的一些有用寄存器（基地址为 0x47510000）：

名称	Bank	字	内存偏移量为 0x47510000	描述
BOOT_CFG0		3	0 0x60	启动 fuse 设置
BOOT_CFG1		3	1 0x64	启动 fuse 设置
BOOT_CFG2		3	2 0x68	启动 fuse 设置
BOOT_CFG3		3	3 0x6c	启动 fuse 设置
MAC1_ADDR	39	3	0x4ec	包含 ENET0 MAC 地址的低 32 位
MAC1/2_ADI	39	4	0x4f0	包含 ENET0 MAC 地址的高 16 位和 ENET1 MAC 地址的低 16 位
MAC2_ADDR	39	5	0x4f4	包含 ENET1 MAC 地址的高 32 位

关于 OCOTP_CTRL 中的熔丝与启动/mac/... 配置之间的完整列表和详细映射，请参阅 i.MX 93 安全参考手册中的“熔丝映射”部分。

7.23.1 在 uBoot 中读取熔丝值

MAC1_ADDR:

```
u-boot=> fuse read 39 3
```

7.23.2 在 Linux 中读取熔丝值

要访问 Linux 中的熔丝内容，NXP 提供了 NVMEM_IMX_OCOTP 模块。所有内存映射的 shadow 寄存器的熔丝内容可以通过 sysfs 访问：

```
target:~$ hexdump /sys/devices/platform/soc\@0/47510000.efuse/fsb_s400_fuse0/nvmem
```

7.23.3 烧录 MAC 地址

假设我们想要烧录以下 MAC 地址：

MAC1	12:34:56:78:90:AA
MAC2	Bb:Cc:Dd:Ee:Ff:D0

我们将在 u-boot 中执行这个：

```
u-boot=> fuse prog 39 3 0x567890Aa
u-boot=> fuse prog 39 4 0xFfD01234
u-boot=> fuse prog 39 5 0xBbCcDdEe
```

7.23.4 烧写启动 fuse

警告

fuse 只能写入一次！如果烧录了错误的启动配置，您可能会轻易地将您的板子变砖。这个过程是不可逆的！

可以在附带的名为 **i.MX93_Fusemap.xlsx** 的电子表格中查找应使用哪个熔丝库/字来编程 BOOT_CFGX，这些信息可以在 *i.MX 93 Applications Processor Reference Manual* 中找到。

这些值应该写入 BOOT_CFG0，可以从第 3 Bank 的第 0 字中读取/写入 fuse。

启动设备	BOOT_CFG0
eMMC	0x20020002
SD 卡	0x20000103

要设置内部 fuse 以从 eMMC 启动，可以使用以下方法进行编程：

```
u-boot=> fuse prog 3 0 0x20020002
```

在这个例子中，我们：

- 将 Boot_Mode 设置为 0b0010 （eMMC），使用 BOOT_CFG0[3:0]，
- 将 eMMC 总线宽度设置为 0b01（8 位），使用 BOOT_CFG0[18:17]。
- 使用 BOOT_CFG0[29] 设置 BT_FUSE_SEL（Boot fuses already programmed）位。

确保通过阅读 *i.MX 93 Applications Processor Reference Manual* 中的 **Boot Fusemap** 章节来设置正确的位。

7.24 TPM

phyBOARD-Nash i.MX 93 配备了可信任的平台模块（TPM），提供基于硬件的安全功能。

以下是一些与 TPM（受信任的平台模块）相关的有用示例

使用 TPM2 工具生成 4 字节随机值：

```
target:~$ tpm2_getrandom --hex 4
```

使用 OpenSSL 工具生成 4 字节随机值：

```
target:~$ openssl rand -engine libtpm2tss --hex 4
```

生成 RSA 私钥并验证其内容：

```
target:~$ openssl genrsa -engine libtpm2tss -out /tmp/priv_key 512
Engine "tpm2tss" set.
target:~$ openssl rsa -check -in /tmp/priv_key -noout
RSA key ok
target:~$ cat /tmp/priv_key
-----BEGIN PRIVATE KEY-----
MIIBVQIBADANBgkqhkiG9w0BAQEFAASCAT8wggE7AgEAAkEAxsvmcbxjwuKnYeuZ
2AVBmuLvYyqF/LpYOD3IB/v+YvEolxdGGmjiFLECU6xZ1j3+dIt4Y1zbcKS1OcWT
I8mbSwIDAQABAkBoy8wrYNhmP/1kzUJIclznPYJckGoZlFI1M7xjGSA9H1xDK6if
5g5CYCHPrbBp8e0mEokPRZoihxxzGTxGPiahAiEA/7OYMOpVZ5SD3YcRsWcQlkWI
MOSPUYg6vxvGG9xp4FcCIQDHB01RoHr+qXJwxIu3/3oQAUBI4ACJ4JRp0KelwhC0
LQIhANJzSvg/dak5l8pU55/99q3nbm7nPnnZSJiP0F6P62gjAiEAjf7qrfMF7Uyt
RkEjwbl2t5Z868FNARGGMVxZT4x+aF0CIGxlmP2pL8xFu1bWB282LSedqZUdQwel
Lxi7+svb2+uJ
-----END PRIVATE KEY-----
```

备注

如果您打算将这些密钥用于任何安全目的，请不要共享您的私有 RSA 密钥。

生成 RSA 公钥并验证其内容：

```
target:~$ openssl rsa -in /tmp/test_key -pubout -out /tmp/pub_key
writing RSA key
target:~$ openssl pkey -inform PEM -pubin -in /tmp/pub_key -noout
target:~$ cat /tmp/pub_key
-----BEGIN PUBLIC KEY-----
MFwwDQYJKoZIhvcNAQEBBQADSwAwSAJBAMbL5nG8Y8Lip2HrmdgFQZri72Mqhfy6
WDg9yAf7/mLxKJcXRhpo4hSxAlOsWdY9/nSLeGNc23CktTnFkyPJm0sCAwEAAQ==
-----END PUBLIC KEY-----
```

设备树 TPM 配置可以在这里找到：https://git.phytec.de/linux-imx/tree/arch/arm64/boot/dts/freescale/imx93-phyboard-nash.dts?h=v6.1.55_2.2.0-phy3#n151

CHAPTER 8

i.MX 93 M33 Core

除了 Cortex-A55 核心外，SoC 中还集成了一个 Cortex-M33 Core 作为微控制器（MCU）。我们的 Yocto-Linux-BSP 在 A55 核心上运行，而 M33 Core 可以作为辅助核心使用，执行额外的任务，采用裸机固件。两个核心都可以访问相同的外设，因此在 M33 Core 的固件或 Linux 操作系统的设备树中，需要限制外设的使用。

我们的 Yocto-BSP 包含来自 NXP 的 M33 Core 预编译固件示例。

本节描述了如何在 phyBOARD-Segin/Nash i.MX 93 上运行预编译的 M33 Core 固件示例。

8.1 运行 M33 Core 示例

运行 M33 Core 固件示例有两种方式：从 U-Boot bootloader 和从正在运行的 Linux 中的 Remoteproc 子系统。

要接收调试信息，请在您的主机 PC 上启动您喜欢的终端软件（例如 Minicom、Tio 或 Tera Term），并将其配置为 115200 波特率、8 个数据位、无奇偶校验和 1 个停止位（8n1），且不使用握手。

在 phyBOARD-Segin/Nash i.MX 93 上需要一个外部的“USB TTL 转串行适配器”。适配器的 I/O 引脚应能够在 3.3V 电压水平下工作。

根据下表，将外部“USB TTL 串行适配器”信号连接到板上的*X16* 连接器：

USB-TTL 适配器引脚	X16 信号	X16 引脚
RXD	X_UART2_TX	5
TXD	X_UART2_RX	8
GND	GND	4

8.1.1 从 U-Boot 运行示例

要使用 U-Boot bootloader 加载固件示例，可以使用 `bootaux` 命令：

1. 准备一张烧写了 BSP 镜像的 SD 卡
2. 通过按任意键停止自动启动

3. 列出 SD 卡第一分区上可用的 |mcore| 固件示例：

```
u-boot=> ls mmc 1
```

备注

可用的固件示例以 `imx93-11x11-evk_m33_TCM_*` 开头，以 `*.bin` 结尾。这些示例来自 NXP 的 Yocto 层 meta-imx，并根据与 phyBOARD-Segin/Nash i.MX 93 硬件的兼容性进行选择。

1. 加载所需的固件示例：

```
u-boot=> fatload mmc 1 ${loadaddr} <firmware.bin>
u-boot=> cp.b ${loadaddr} 0x201e0000 ${filesize}
u-boot=> run prepare_mcore
u-boot=> bootaux 0x1ffe0000 0
## Starting auxiliary core addr = 0x1FFE0000...
```

程序的输出应显示在 |mcore| 的调试 UART 上。

8.1.2 通过 Remoteproc 在 Linux 上运行示例

Remoteproc 是一个模块，它允许您在运行时从 Linux 控制 M33 Core 。可以加载 M33 Core 的固件示例，并在 Linux 中启动或停止执行。要使用 Remoteproc，需要设置设备树 overlay：

在目标设备的 `/boot` 目录中编辑 `bootenv.txt` 文件，添加 imx93-phycore-rpmsg.dtso ：

```
overlays=imx93-phyboard-segin-peb-av-02.dtbo imx93-phycore-rpmsg.dtso
```

重启目标并在 U-Boot 中执行:

```
u-boot=> run prepare_mcore
```

针对 M33 Core 的固件示例 `*.elf` 文件可以在 `/lib/firmware` 下找到。列出可用的固件示例：

```
target:~$ ls /lib/firmware/*.elf
```

要加载固件，请输入：

```
target:~$ echo /lib/firmware/<firmware>.elf > /sys/class/remoteproc/remoteproc0/firmware
target:~$ echo start > /sys/class/remoteproc/remoteproc0/state
```

要加载不同的固件，M33 Core 需要停止：

```
target:~$ echo stop > /sys/class/remoteproc/remoteproc0/state
```

备注

在目标设备的 `/lib/firmware` 目录中找到的例子来自 NXP 的 Yocto 层 meta-imx，并根据与 phyBOARD-Segin/Nash i.MX 93 硬件的兼容性进行选择。

NXP 的一些固件示例需要加载额外的 Linux 内核模块。

例如，当加载 `imx93-11x11-evk_m33_TCM_rpmsg_lite_str_echo_rtos.elf` 固件时，需要加载相应的 `imx_rpmsg_tty` 模块：

```
target:~$ modprobe imx_rpmsg_tty
```

这将一个 RPMsg 端点作为虚拟 TTY 暴露在 `/dev/ttyRPMSG30` 。现在可以通过输入以下内容从 A55 核心发送消息到 M33 Core：

```
target:~$ echo "PHYTEC" > /dev/ttyRPMSG30
```

观察 M33 Core 调试 UART 应该会产生以下输出：

```
RPMSG String Echo FreeRTOS RTOS API Demo...

Nameservice sent, ready for incoming messages...
Get Message From Master Side : "PHYTEC" [len : 6]
```